

Liquidity Party

A Multi-Asset Logarithmic Market Scoring Rule AMM Deployment Whitepaper

June 11, 2026

Abstract

Liquidity Party is a permissioned-deploy, non-upgradeable multi-asset automated market maker. The pricing kernel is the Logarithmic Market Scoring Rule (LMSR) with the liquidity parameter quasi-statically tied to pool size as $b(\mathbf{q}) = \kappa S(\mathbf{q})$, but held *fixed for the duration of each block* and anchored to $\min(\sigma_{\text{swap}}, \sigma_{\text{live}})$, where σ_{swap} is a block-stepped EMA of the previous block’s end-state $S(\mathbf{q})$ (§3.2). This per-block freeze is the load-bearing component of a novel defense against *wedge attacks* — mint sandwiches and JIT minting that exploit the convexity of LMSR against the linearity of proportional mint and burn (§3). The defense combines a raw single-block $\Delta\sigma_q$ deviation gate, a min-clamped proportional burn that structurally closes JIT extraction, a per-window rate limit on the proportional mint-resize factor γ (the fraction by which a mint grows the pool, equal to the new LP minted divided by the pre-mint supply), and a post-mint LP lock. Together these allow LMSR pricing to coexist with the standard AMM treatment of LP tokens as pro-rata claims on pool reserves. Pool parameters are immutable post-deployment; the only mutable admin levers on a live pool are the protocol-fee recipient and the irreversible `kill()` switch. The admin cannot mint LP, change fees, freeze users, or move pool reserves — a leaked admin key presents little to no danger to LPs (§5.4). Pool creation is restricted to the trusted operator, who is responsible for upstream token vetting via a documented validator pipeline. Where the implementation makes sub-ulp rounding choices, every choice is biased toward existing LPs and against takers and the protocol developers; the protocol’s primary client is the LP. This document specifies the pricing kernel, the wedge-attack defense, the liquidity operations, the production fee structure, the admin and deployment surface, and the security-review methodology applied prior to launch.

Contents

1	Overview	3
2	Pricing Kernel	5
2.1	LMSR Cost Function and the $\kappa S(\mathbf{q})$ Parameterization	5
2.2	Inventory Convention: The Convexity Flip vs Hanson’s LMSR	5
2.3	Marginal Prices	7
2.4	Block-Anchored b via $\min(\sigma_{\text{swap}}, \sigma_{\text{live}})$	7
2.5	Swap Mapping	8
2.6	LP-Safety Property of the Block-Anchored Kernel	8
2.7	Slippage Protection	9

3	Wedge-Attack Defense	9
3.1	Defense Architecture	10
3.2	State and EMA Update Rules	10
3.3	The Gate	12
3.4	Min-Clamped Burn and Why It Closes JIT	13
3.5	Per-Window Rate Limit	14
3.6	Post-Mint LP Lock	14
3.7	Boundary Attack and the Swap-Fee Coupling	15
3.8	Synchronous Semantics and the Optional Mint Queue	15
3.9	Attack Surfaces Closed and Residual Surfaces	15
4	Liquidity Operations	16
4.1	Proportional Mint	17
4.2	Proportional Burn	17
4.3	Single-Asset Mint: <code>swapMint</code>	18
4.4	Single-Asset Redeem: <code>burnSwap</code>	19
5	LP Protection Mechanisms	20
5.1	Kernel-Level LP Safety	20
5.2	Fee and Protocol-Share Rounding	20
5.3	Correlated-Asset LVR Reduction (Conditional)	21
5.4	Admin-Side LP Protection	21
5.5	Burn Is Always Live	21
6	Fee Structure	22
6.1	Per-Asset Swap Fees	22
6.2	Compound-Operation Fees on <code>swapMint</code> and <code>burnSwap</code>	22
6.3	Protocol Fees	23
6.4	Rounding Policy	23
7	Funding Modes	24
8	Contract Architecture	25
8.1	PartyPool: Proxy and Storage	25
8.2	PartyPlanner: Factory and Registry	25
8.3	PartyInfo: View Helpers	25
8.4	PartyConcierge: Router	25
8.4.1	Mint Queue and Keeper Mechanism	26
9	Admin Powers and Parameter Fixity	27
9.1	Mutable State on a Live Pool	27
9.2	Emergency Kill Switch	28
9.3	What the Operator Cannot Do	28
10	Deployment Guidelines	28
10.1	Operator Workflow	28
10.2	Token-Selection Criteria: Correlated-Asset LVR Reduction	29
10.3	Choosing κ	30
10.4	Bases β_i and Market-Calibrated Initialization	30

10.5 Fee Calibration	31
10.6 Native-Asset Wrapping	32
11 Security Review Process	33
11.1 Layered Defense	33
11.2 Internal Review Artifacts	33
11.3 Tooling	34
11.4 External Audit	34
11.5 Bug Bounty	34
11.6 Vulnerability Disclosure	34
11.7 Out of Scope	35
12 Risk Management	35
12.1 Token-Slot Drainage Is Expected LP Behavior	35
12.2 LP Loss Bound	36
12.3 Liquidity-Manipulation Resistance	36
12.4 Numerical Envelope	36
12.5 Operational Mitigations	36
12.6 Out-of-Scope Token Classes	37
13 Deployment Surface	37
14 Disclaimers	38
References	38

1 Overview

Liquidity Party is a multi-asset AMM that holds many tokens in a single pool and quotes every pair within that pool from a single convex potential. The system is designed for capital efficiency on long-tail asset baskets: an n -asset pool exposes $n(n - 1)/2$ pairs with one set of LP shares, eliminating the fragmentation cost of pairwise CFMM deployments. Per-operation gas grows with the basket size (the LMSR potential touches every slot), so the practical range is roughly $n \leq 50$ tokens per pool; approximate gas costs for production-typical sizes are summarized in Table 1.

Table 1: Approximate gas costs by pool size

Assets	Pairs	Swap	Mint	Burn	SwapMint	BurnSwap
2	1	138,000	174,000	143,000	173,000	144,000
10	45	150,000	473,000	424,000	388,000	349,000
20	190	165,000	847,000	776,000	655,000	608,000
30	435	180,000	1,220,000	1,130,000	923,000	859,000
50	1225	210,000	1,970,000	1,830,000	1,460,000	1,380,000

System layout. The on-chain system consists of four singleton-or-immutable contracts:

- **PartyPlanner** — factory and pool registry. `onlyOwner newPool` deploys each **PartyPool** via `CREATE2` with the caller-specified configuration vector: $(\kappa, \mathbf{f})$, the wedge-defense immutables (gate threshold `MINT_DEVIATION_PPM`, `EMA_SHIFT_BLOCKS`, the per-window γ cap `MAX_GAMMA_PER_WINDOW_PPM`, and `MINT_LOCK_BLOCKS`), and the protocol-fee settings. The planner stores no default parameters of its own (§8). Indexes pools by token for off-chain discovery.
- **PartyPool** — the pool itself, holding token reserves and issuing an ERC-20 LP share token. The pool is a non-upgradeable `DELEGATECALL` proxy over two implementation libraries (`PartyPoolExtraImpl1`, `PartyPoolExtraImpl2`) due to the EIP-170 24 KB contract-size cap.
- **PartyInfo** — read-only view contract for prices, quotes, and denominators. Pricing helpers are exposed here rather than on the pool so that **PartyPool** fits its size budget.
- **PartyConcierge** — user-facing router. Takes token addresses rather than indices, supports a single ERC-20 approval covering every pool, and exposes an EIP-7730 clear-signing surface. Funds pools via the callback funding mode (see §7), and offers an optional keeper-executed mint queue for rate-limited mints (§8.4.1).

The pool’s per-token configuration (bases and per-asset fee rates) is stored in an `SSTORE2`-style data contract referenced by an immutable address in the pool. Hot paths read it via `EXTCODECOPY`; no SLOAD on the bases or fees occurs on the swap path.

Notation. We index assets by $i \in \{0, \dots, n-1\}$ and write the pool’s internal inventory as $\mathbf{q} = (q_0, \dots, q_{n-1})$, with the size metric $S(\mathbf{q}) = \sum_i q_i$. The liquidity parameter is $b(\mathbf{q}) = \kappa S(\mathbf{q})$ for a positive constant κ fixed at deployment. Internal quantities are maintained in Q64.64 fixed-point (`ABDKMath64x64`); user-facing token amounts are integer base units (`uint256`) and are converted on the boundary using per-asset denominators β_i (the *bases*) set at deployment: $q_i = \text{balance}_i / \beta_i$.

Primary audience: liquidity providers. Liquidity Party’s first concern is the LP. The protocol is engineered so that every decision the LP cannot make on their own behalf — rounding, approximation, cap-and-invert branch selection, fee/protocol-share split, emergency response — is biased in favor of the LP. Takers and integrators are first-class users of the protocol surface but inherit the LP-favor invariants from the LP side; they should design assuming that any sub-ulp ambiguity in the rounding chain will resolve in the LP’s favor and against their own position.

Reading guide. §2 specifies the pricing kernel and its block-anchored b . §3 specifies the wedge-attack defense (the raw single-block $\Delta\sigma_q$ gate, the min-clamped burn, the per-window γ rate limit, and the post-mint LP lock), which is the novel mechanism that lets LMSR pricing coexist with pro-rata LP accounting. §4 specifies the user-facing liquidity operations (proportional mint, proportional burn, `swapMint`, `burnSwap`) as they interact with the defense. §5 consolidates the LP-favor guarantees that the kernel and the defense establish. §6–§7 describe fees and the four funding modes. §8 covers the deployed contract surface, §9 the admin powers and parameter fixity, and §10 the operator’s deployment-time calibration choices. §11 documents the security review process that gates production launch. §12 states the residual risk surface and operational mitigations, including the framing of token-slot drainage as expected and industry-normal LP behavior.

2 Pricing Kernel

2.1 LMSR Cost Function and the $\kappa S(\mathbf{q})$ Parameterization

The pool maintains the LMSR cost function in *inventory convention*

$$C(\mathbf{q}; b) = -b \ln \left(\sum_{k=0}^{n-1} e^{-q_k/b} \right), \quad (1)$$

where larger q_i corresponds to a more abundant slot and therefore a *cheaper* marginal price. The liquidity parameter b tracks pool size as $b = \kappa S(\mathbf{q})$ but is held *fixed for the duration of each block* and anchored to the size metric computed at the end of the previous block (§2.4). This quasi-static LS-LMSR treatment retains the three properties that drive the rest of the design:

1. **Scale invariance.** A proportional rescaling $\mathbf{q} \mapsto \lambda \mathbf{q}$ that also rescales b by λ leaves every pairwise marginal price unchanged, so proportional mint and burn are value-neutral at matched b .
2. **Bounded LP loss.** The classical LMSR worst-case-loss $b \ln n$ becomes $\kappa S(\mathbf{q}) \ln n$ at the current state; per unit of size this is $\kappa \ln n$, fixed by the deployment parameters.
3. **Liquidity sensitivity.** As the pool absorbs trades and accrues fees across blocks, $S(\mathbf{q})$ grows and the next block’s anchor grows with it; depth deepens automatically in proportion to TVL without governance intervention.

The continuous LS-LMSR construction of Othman et al. (2013) admits a similar size-dependent b but with a continuously evolving liquidity trajectory whose exact pricing requires integral evaluation. Our $b(\mathbf{q}) = \kappa S(\mathbf{q})$ choice is a fixed proportionality that is sampled once per block from a block-end snapshot, eliminating the path-integral dependence within any single transaction while retaining liquidity sensitivity across blocks.

2.2 Inventory Convention: The Convexity Flip vs Hanson’s LMSR

Liquidity Party’s pricing kernel is mathematically a sign-and-curvature reflection of Hanson’s original LMSR cost function. This subsection states the relationship explicitly because it is easy to misread the formula above as Hanson’s cost function with a notational substitution. It is not: it is a different cost surface that supports a different agent.

Hanson’s outstanding-shares convention. Hanson (2002) defines the LMSR cost function over a vector $\mathbf{s} = (s_0, \dots, s_{n-1})$ of *outstanding shares*, the cumulative quantity of each outcome-share the market maker has sold into the market:

$$C_H(\mathbf{s}; b) = +b \ln \left(\sum_{k=0}^{n-1} e^{+s_k/b} \right). \quad (2)$$

The cost function is *convex* in $\mathbf{s}$. Its gradient $\partial C_H / \partial s_i = e^{s_i/b} / \sum_k e^{s_k/b}$ is the marginal price the market maker charges to sell one additional share of outcome i : *more abundant outstanding shares of i imply a higher marginal price for that outcome*. The economic agent is a market-maker seat that is *short* the basket — it sells shares into the market and collects payments at the gradient price. Its worst-case running cost is bounded by $b \cdot \ln n$. Hanson’s LMSR is the canonical pricing kernel for prediction markets, where the market-maker seat is paid for information aggregation rather than for holding inventory.

Liquidity Party’s inventory convention. A standard AMM does the opposite job. There is no market-maker seat; the protocol holds a pool of n asset reserves $\mathbf{q} = (q_0, \dots, q_{n-1})$ and an LP-token holder is *long* that basket pro-rata. A trader pays asset i *into* the pool (raising q_i) and pulls asset j *out* (lowering q_j); the marginal price along that exchange should fall as q_j shrinks and the pulled asset becomes scarce. We need a cost surface under which q_i is inventory, not outstanding shares, with the correct “more abundant slot is cheaper” direction on the marginal price.

The naive route — substitute $s_i := -q_i$ into (2) — only flips the gradient direction; it leaves the outer sign of C_H positive and the function convex in $\mathbf{q}$. The convexity-in- $\mathbf{q}$ would mean the cost function *grows* when the pool gets larger via a proportional deposit, contradicting the desired LP-side accounting (a deposit shouldn’t read as a cost incurred). To get an inventory cost surface whose gradient is correct *and* whose curvature matches the LP-as-long-the-basket geometry, we flip *both* signs:

$$C(\mathbf{q}; b) = -b \ln \left(\sum_{k=0}^{n-1} e^{-q_k/b} \right). \quad (3)$$

This is *not* the same function as (2) under relabeling. The inner exponent is negated *and* the outer sign is flipped — a double flip. The result is *concave* in $\mathbf{q}$, while Hanson’s is convex in $\mathbf{s}$. The gradient $\partial C / \partial q_i = e^{-q_i/b} / \sum_k e^{-q_k/b}$ is the marginal weight assigned to slot i , with $\partial C / \partial q_i$ *decreasing* in q_i — exactly the “more abundant slot is cheaper” direction we want, and exactly the inverse of Hanson’s direction.

Two flips, not one. The pairing of *input–output direction flip* ($s_i \leftrightarrow -q_i$, “shares-sold” becoming “inventory-held”) with *curvature flip* (convex C_H becoming concave C) is what distinguishes our formulation from a notational relabel of Hanson:

- The **input/output flip** reverses the semantic role of the state vector. In Hanson, s_i rises when the AMM *sells* into the market; in our convention, q_i rises when a trader sells asset i *into* the pool. The same positive Δ on the state vector means opposite cash-flow direction on the protocol side.
- The **curvature flip** reverses the role of the cost function. Hanson’s C_H is the market-maker’s cumulative *liability* to share-holders, bounded above by $b \ln n$; our C is a concave potential whose level sets define the cost-preserving swap manifold. The LP-side worst-case loss bound is still $b \ln n$, but it arrives from the opposite direction — the maximum amount of value that can be *drained* from a pro-rata-long pool by an adversarial price path, rather than the maximum payout the Hanson seat can owe.

Cost preservation under inventory convention. A swap of a units of asset i in for y units of asset j out is value-neutral on the LP side if and only if it preserves the inventory cost:

$$C(\mathbf{q} + a \mathbf{e}_i - y \mathbf{e}_j; b) = C(\mathbf{q}; b). \quad (4)$$

The closed-form solution at fixed b is the Hanson exact-input mapping rewritten in inventory variables:

$$y = b \ln \left(1 + r_0 (1 - e^{-a/b}) \right), \quad r_0 = e^{(q_j - q_i)/b}. \quad (5)$$

Note the position of the minus signs: the marginal-price ratio r_0 is keyed to $(q_j - q_i)$ (output minus input), and the input contribution is $e^{-a/b}$ (input depressing the output rate as it floods slot i). These signs are the visible footprint of the double flip in the cost surface; they would all run the other way under Hanson’s outstanding-shares convention.

The convexity flip under LS-LMSR. Othman et al. (2013) extend LMSR to liquidity-sensitive by letting b track a function of state, typically $b = \kappa \cdot S(\mathbf{q})$. Under Hanson’s convention this keeps C_H convex; the larger $S(\mathbf{q})$ flattens the convexity, deepening liquidity. Under our inventory convention the same parameterization keeps C concave, and the larger $S(\mathbf{q})$ flattens the concavity, deepening liquidity. The LS extension respects the convention — it does not introduce a third flip. The liquidity parameter $b = \kappa \cdot \min(\sigma_{\text{swap}}, \sigma_{\text{live}})$ (§2.4) carries the same “larger b is deeper liquidity” meaning that Hanson and Othman et al. attach to it, but it parameterizes a concave surface rather than a convex one.

Why this matters operationally. The convexity flip is invisible to a trader who only reads the marginal-price formula $P(i \rightarrow j) = \exp((q_j - q_i)/b)$. That formula has the same shape as Hanson’s $P_H(s_i)$, just with a different sign on the state argument. The flip becomes visible whenever the LP side of the accounting is exercised: proportional mint, proportional burn, the LP-loss bound, the wedge-attack defense (§3), and the cost-preservation invariant the kernel solves. Throughout this document, when we write “LMSR cost function,” “LS-LMSR,” or “cost-preservation,” the quantity meant is the inventory-convention C of (3) — not Hanson’s C_H .

2.3 Marginal Prices

With b held constant within a block, the gradient of C recovers softmax-style prices and the instantaneous marginal exchange rate for swapping input $i \rightarrow$ output j is

$$P(i \rightarrow j \mid \mathbf{q}, b) = \exp\left(\frac{q_j - q_i}{b}\right). \quad (6)$$

A more abundant output slot (larger q_j) buys more output tokens per unit input. Per-pair denomination-adjusted prices are exposed via `PartyInfo.price(pool, i, j)` as a Q128.128 fixed-point quantity that accounts for the bases β_i, β_j .

2.4 Block-Anchored b via $\min(\sigma_{\text{swap}}, \sigma_{\text{live}})$

Every pricing read on the hot path — the swap kernel, the per-leg quotes inside `swapMint` and `burnSwap`, the `PartyInfo` view helpers — computes

$$b = \kappa \cdot \min(\sigma_{\text{swap}}, \sigma_{\text{live}}), \quad (7)$$

where $\sigma_{\text{live}} = \sum_i q_i$ is the live size metric recomputed from the current pool inventory, and σ_{swap} is an EMA-anchored estimate of the previous block’s end-state σ_{live} (the full state machine and update rules are specified in §3.2). The minimum ensures that any mid-block expansion of σ_{live} — whether from arbitrage vig, proportional mint, or attacker skew — does not retroactively widen the depth the rest of the block prices against; a block can only *shrink* the effective b , never grow it.

Why fixed- b within a block. Recomputing b per leg of a multi-step operation (`swapMint`, `burnSwap`) from the evolving local state $\mathbf{q}_{\text{local}}$ would produce three mutually inconsistent pricing surfaces across `swap`, `swapMint`, and `burnSwap`, and closed cycles combining the three operations could extract LP value across the inconsistency. Anchoring b to the same block-aligned $\min(\sigma_{\text{swap}}, \sigma_{\text{live}})$ for every leg of every operation collapses the three surfaces into one and closes the cycle exploit; combined with the LP-favor rounding policy (§6.4), the residual asymmetry between the operations is consistently in the LP’s favor.

Why min, not σ_{swap} directly. A pool that has been recently *drained* by a proportional burn sees σ_{live} drop below σ_{swap} (the EMA still reflects the larger pre-burn state). Pricing against the stale σ_{swap} would let an attacker swap into the shrunken pool at the pre-burn depth — buying at a liquidity depth the pool no longer has, and extracting the difference from the remaining LPs. The min closes that surface: post-burn the pool prices at the shrunken σ_{live} , and post-skew-swap it prices at the unmoved σ_{swap} — both directions key to the *lower* of the two, which is uniformly LP-favoring.

2.5 Swap Mapping

Let i, j be the input and output asset indices and let a be the internal input passed to the kernel. The production swap kernel (`LMSRKernel.swapAmountsForExactInput`) evaluates the single-pass Hanson exact-input formula at the block-anchored b :

$$\begin{aligned} b &= \kappa \cdot \min(\sigma_{\text{swap}}, \sigma_{\text{live}}), \\ r_0 &= \exp((q_j - q_i)/b), \\ y &= b \ln(1 + r_0(1 - e^{-a/b})). \end{aligned} \tag{8}$$

The implementation caps output at the available inventory q_j when the inner term of $\ln$ degenerates (cap-and-invert branch). There is no path-integral correction inside the kernel: the block-fixed b already provides the round-trip invariant the kernel needs, and any intra-block closed sequence of `swap`, `swapMint`, and `burnSwap` prices against a single consistent surface. Fees (§6) and LP-favor rounding (§6.4) make every fee-aware closed cycle strictly loss-making for the attacker, and the wedge defense (§3) closes the LMSR-convexity attack on mint and burn that the block-fixed surface would otherwise leave open.

2.6 LP-Safety Property of the Block-Anchored Kernel

The path-exact reference is the liquidity-sensitive LMSR mapping under inventory convention: the y^* that solves the cost-preservation equation

$$C(\mathbf{q} + a e_i - y^* e_j; b^*) = C(\mathbf{q}; b^*), \tag{9}$$

under a continuously evolving $b^* = \kappa S(\mathbf{q})$. With b truly state-dependent, both b and the inner exponents change as the state moves from $\mathbf{q}$ to $\mathbf{q} + a e_i - y e_j$, so y^* has no closed form. Our implementation freezes b at $\kappa \min(\sigma_{\text{swap}}, \sigma_{\text{live}})$ for the duration of the block, with σ_{live} at most the pre-trade size metric on the first state change of the block; subsequent mid-block reads see the same or smaller σ_{live} value (via the min), so the effective b used during a swap is at most the exact-LS-LMSR midpoint trajectory b .

LP-safety property. *For every $(a, \mathbf{q}, \kappa, n)$ in the production envelope, the kernel output y satisfies*

$$y \leq y^*.$$

Equivalently, the swapper receives at most what the path-exact LS-LMSR would pay, and the LP loses at most what the path-exact LS-LMSR would surrender. The proof is the inequality on b above: the block-fixed b never exceeds the path-exact b -trajectory, the Hanson exact-in mapping is increasing in b on the relevant regime, and therefore the kernel's y is below the path-exact y^* .

Empirical zero-extraction. The complementary guarantee — that no closed *sequence* of legs (e.g. `swap`, `swapMint`, `burnSwap`, `mint`, `burn`) can accumulate sub-ulp residue into an extractable round trip — is addressed by the adversarial cycle test suite in `test/SwapMintBurnSwapBAnchorExploit.t.sol` and the skew-attack suites (`ArbHarness`, `StandardPoolAttacks`, `PoC_WedgeAttack`). The suites run closed cycles against pools already driven to imbalance, covering 2-leg pure-swap loops, 3- and 4-leg mixed swap / `swapMint` / `burnSwap` / `mint` / `burn` loops, and intra-block self-sandwich cycles. The assertion at every cycle boundary is *strict*: any closed cycle that leaves the attacker even **1 wei** richer (measured at pre-cycle marginal prices) is a test failure, and the same strict bound is required to hold on the pool TVL at pre-cycle prices. **No grid point fails.** The shared-block-*b* surface makes the intra-block cycle pricing self-consistent; the wedge defense (§3) closes the mint-side convexity attack on top of that.

Capacity caps. For output y that would exceed available q_j , the kernel returns $y = q_j$ at the cap-and-invert boundary. The inverse mapping (`amountInForExactOutput`) returns the input required for a target output, using

$$a(y) = b \ln\left(\frac{r_0}{r_0 + 1 - e^{y/b}}\right), \quad (10)$$

which is the exact closed-form inverse of the Hanson exact-in mapping (used internally on the cap-and-invert branch and on the inverse leg of `swapMint`, not as a public exact-out swap entry point).

Numerical stability. The kernel computes ratios like $r_0 = \exp((q_j - q_i)/b)$ directly rather than dividing exponentials, guards every `exp` and `ln` call to bounded domains (`EXP_LIMIT = 32.0`), and checks the positivity of the inner `ln` argument before evaluation. Domain violations revert; outputs are clamped to capacity rather than extrapolated. The cost function evaluates log-sum-exp with one-pass dynamic recentering for stability under dispersed q_k .

2.7 Slippage Protection

Every user-facing swap accepts a `minAmountOut` (or, on `swapMint`, a `maxAmountIn`). If the computed output is below `minAmountOut`, the call reverts. This is denomination-aware slippage protection requiring no price-unit conversions. Off-chain callers that prefer a price-based bound may use the bisection helper `PartyInfo.swapAmountsForExactPrice` to convert a target post-swap forward price into a (`maxAmountIn`, `minAmountOut`, `fee`) triple suitable for direct submission to `swap()`.

3 Wedge-Attack Defense

LMSR pricing is convex; proportional mint and burn are linear in pool size. The mismatch between the two creates a structural attack family we call the *wedge*: an attacker front-runs a victim’s mint with a skew swap that moves the pool to an off-market state, the victim’s proportional deposit deepens the pool at the manipulated price ratio, the attacker back-runs with a reverse swap that captures the convexity-bonus value that the deposit added to the wedge. The attacker extracts face value from both the victim and the pre-existing LPs.

This attack is the central obstacle to running LMSR pricing alongside a standard AMM-style LP token, where LP shares are pro-rata claims on pool reserves. Hanson’s original LMSR is operated by a market-maker seat, not by pro-rata LPs; the convexity belongs to the seat, not to depositors,

so the wedge has no mint/burn surface to attack. Liquidity Party retains LMSR’s marginal-price geometry while issuing pro-rata LP tokens, which re-opens the wedge surface. The defense specified in this section is what allows that combination.

A full design and parameter justification is in `doc/rate-limited-mints.md`; this section gives the production-load-bearing summary.

3.1 Defense Architecture

Four components, plus a load-bearing coupling to the pool’s swap fee:

1. **The raw single-block $\Delta\sigma_q$ gate.** A single Q64.64 slot, `_prevBlockEndSigmaQ`, holds the $\sigma_q = S(\mathbf{q})$ snapshot captured at the first state change of the current block — the end-of-previous-block value, by construction. The gate trips and `mint / swapMint` revert while

$$|\sigma_{\text{live}} - \sigma_{\text{prevBlockEnd}}| \geq \tau \cdot \sigma_{\text{prevBlockEnd}},$$

where τ is the immutable per-pool gate threshold (the `MINT_DEVIATION_PPM` parameter, τ_d). The reference is a fixed block-*start* snapshot, not in-block state, so spam-fill within a block cannot manipulate it — all in-block activity is measured against the same anchor. Crucially the gate measures only the move *within this block*: it trips inside a block that itself carries a $\geq \tau_d$ jump and reopens the next, rather than staying tripped while a slow EMA catches up (§3.3, “Why raw, not σ_{swap} ”).

2. **Min-clamped proportional burn.** The proportional-burn payout is scaled by $\alpha' = \alpha \cdot \min(\sigma_{\text{swap}}, \sigma_{\text{live}}) / \sigma_{\text{live}}$ rather than pure α . When σ_{swap} lags σ_{live} — as it does immediately after a swap added vig — burners receive their *pre-swap* pool value, not the post-swap value. The vig stays in the pool for LPs who wait for σ_{swap} to converge. This is what closes JIT minting structurally.
3. **Per-window γ rate limit.** A continuous, EMA-decayed accumulator caps $\sum \gamma$ across all mints within the current EMA window at `MAX_GAMMA_PER_WINDOW_PPM`. This bounds the aggregate gate-boundary residual per window and prevents attacker self-sandwich cycling.
4. **Post-mint LP lock.** Freshly-minted LP is non-burnable for a per-pool immutable window of `MINT_LOCK_BLOCKS` (§3.6). The LP remains freely transferable, but the lock travels with the tokens — a transfer of still-locked LP migrates the lock to the recipient, who cannot burn it until the original unlock block. The lock forces a mint to commit capital across blocks before it can exit, which closes the atomic `mint`→`burn` round-trip that would otherwise exhaust the rate-limit budget at zero net capital cost, and also closes multi-block JIT around large swaps once σ_{swap} has caught up enough that the min-clamp no longer penalizes the burn.

The fourth load-bearing piece is the pool’s per-asset **swap fee** (§6). A linear-in-swap-size fee on the attacker’s two skew-and-reverse legs flips the gate-permitted boundary-attack per-attack net negative; the per-family τ and fee recommendations in `doc/rate-limited-mints.md` are calibrated together against the boundary-attack simulator.

3.2 State and EMA Update Rules

The defense state lives in three storage slots adjacent to the LMSR state in `PartyPoolStorage.PoolState`:

- `_sigmaSwap` (Q64.64): the EMA-smoothed estimate of σ_q . It is *not* the gate reference — it drives only the block-anchored swap b (§2.4) and the burn value clamp (§3.4), where its slow convergence is a feature.
- `_prevBlockEndSigmaQ` (Q64.64): the σ_q snapshot captured at the first state-change of each new block (which is the end-of-previous-block snapshot, by construction). **This is the mint-gate reference** — both the deviation anchor and the denominator (§3.3). It also serves as the EMA target for the σ_{swap} step.
- `_lastUpdateBlock` (uint64): the block number of the last σ_{swap} step.
- `_gammaAccum` (Q64.64): the EMA-decayed accumulator of applied γ for the per-window rate limit.
- `_gammaAccumLastBlock` (uint64): block number of the last accumulator decay update.

The σ_{swap} state evolves by the following rules:

On the first state change of a new block. The pool first *captures* the gate reference $\sigma_{\text{prevBlockEnd}} = \sigma_{\text{live}}$ (read from `qInternal` before any mutation in this call, so it is exactly the end-of-previous-block value), then steps the EMA toward it:

$$\sigma_{\text{swap}} \text{ += } \frac{\sigma_{\text{prevBlockEnd}} - \sigma_{\text{swap}}}{2^{\text{EMA_SHIFT_BLOCKS}}}.$$

The freshly-captured $\sigma_{\text{prevBlockEnd}}$ is the fixed anchor against which every mint in this block is gated, so a plain mint with no prior same-block activity sees $\sigma_{\text{live}} = \sigma_{\text{prevBlockEnd}}$ and passes the gate by construction. The shift is fixed at one shift per block-with-activity, regardless of how many blocks have elapsed since the last update. Quiet pools accumulate no “convergence credit”. After many quiet blocks the σ_{swap} defense is in fact *stronger*, not weaker.

On proportional mint. σ_{swap} , σ_{live} , and the gate reference $\sigma_{\text{prevBlockEnd}}$ all scale by $(1 + \gamma)$. Scaling σ_{swap} preserves the proportional invariant $\sigma_{\text{swap}}/\sigma_{\text{live}}$ for its b-anchor and burn-clamp roles. Scaling the gate reference by the *same* factor is what keeps the raw gate from reading a proportional event as volatility: a proportional grow preserves relative prices, so without this write a second same-block mint would see the first mint’s $(1 + \gamma)$ inventory scaling as a $\Delta\sigma_q$ jump and spuriously trip. The scaling is multiplicative (not an overwrite with σ_{live}), so any genuine same-block skew already in the reference scales along with it and stays caught — the same-block sandwich defense is preserved.

On proportional burn. The pool computes the effective burn fraction α' as defined in §3.1, then scales reserves `q` by $(1 - \alpha')$ (the value actually paid out) while scaling *both* σ_{swap} and the gate reference $\sigma_{\text{prevBlockEnd}}$ by $(1 - \alpha)$ — the *LP-supply* scale, not the reserve scale. The LP-token supply is likewise reduced by the full α that the user requested. Tying the σ_{swap} shrink to $(1 - \alpha)$ rather than $(1 - \alpha')$ is what makes a clamped burn *fragmentation-invariant*: N burns of $\Delta L/N$ total exactly the same payout as one burn of ΔL (§3.4). The gate reference scales by the same $(1 - \alpha)$ for the analogous reason as on mint: a burn shrinks inventory proportionally, so a later same-block mint must not read that shrink as a $\Delta\sigma_q$ jump. When the clamp is inactive ($\alpha' = \alpha$ — the converged, full-drain, or killed cases) the reserve and supply scale factors coincide.

On swap. Only σ_{live} changes (it is derived from $\mathbf{q}$ on demand). No mutation to σ_{swap} happens within a block; the block’s end-state σ_{live} is what feeds the next block’s EMA step.

Non-volatility reconciliation. Two events move σ_{live} without being a manipulation jump, and every LP operation reconciles them into the gate reference so a same-block mint does not read them as in-block volatility. *Retained LP-fee backlog:* a plain **swap** advances $\mathbf{q}_{\text{Internal}}$ by the gross output but retains the LP-fee share off-inventory, so the backlog must be folded back into $\mathbf{q}_{\text{Internal}}$ before the next LP operation gates against it. **burn**, **swapMint**, and **burnSwap** fold it explicitly at their start; **mint** folds it implicitly, by rescaling σ_{swap} across the proportional rebuild (which absorbs the backlog) rather than by the bare $(1 + \gamma)$ factor. When the fold is explicit, $\sigma_{\text{prevBlockEnd}}$ is advanced *additively* by the same σ_{live} delta — additive, not an overwrite, so any genuine prior same-block swap stays measured against the true block-start reference. *Physical donations / over-delivery drift* (balance > cached + owed) are folded by the **mint** / **burn** sweep, rescaling σ_{swap} by the resulting ratio (the hot **swap** and the single-asset variants skip the sweep for gas; their drift is reclaimed by the next **mint** / **burn**). Outside these reconciliations, the block-start capture, and the proportional mint/burn scale, $\sigma_{\text{prevBlockEnd}}$ is never written.

Anti-spam-fill, anti-quiet-pool. The step is keyed on **block.number**, not on the count of in-block transactions or wall-clock time. Spam-filling 256 swaps in a block produces exactly one EMA step. After arbitrary inactivity, the first state change in a new block applies one shift step against the end-of-previous-block snapshot — the EMA does not “cash in” accumulated convergence on a single update. The **block.number** anchor also rules out L2-sequencer **block.timestamp** manipulation.

3.3 The Gate

mint and **swapMint** check the gate at the start of the operation, comparing the *current* (post-swap-leg, for **swapMint**) σ_{live} against the block-start reference $\sigma_{\text{prevBlockEnd}}$:

$$|\sigma_{\text{live}} - \sigma_{\text{prevBlockEnd}}| \cdot 10^6 \geq \text{MINT_DEVIATION_PPM} \cdot \sigma_{\text{prevBlockEnd}} \Rightarrow \text{revert "volatile market"}.$$

$\sigma_{\text{prevBlockEnd}}$ is used as *both* the deviation anchor and the denominator, and the σ_{live} here is the *pre-mint* σ_q : the mint’s own proportional γ growth is deliberately excluded, so the gate asks “did the pool move this block before this mint”, not “does this mint grow the pool”.

The inequality is *non-strict* ($\geq$, not $>$) for an important reason: an attacker can size a **swapMint**’s swap leg to land the pool exactly at $\tau \cdot \sigma_{\text{prevBlockEnd}}$ deviation from the block-start reference. A strict $>$ check would let that exactly-on-threshold poison swap through; non-strict catches the boundary case.

Gate-passing is required but not sufficient: each **mint** also checks **maxAmountsIn** and the rate-limit budget; the operation reverts if either fails.

Why raw, not σ_{swap} . Anchoring this gate on the EMA-smoothed σ_{swap} instead ($|\sigma_{\text{live}} - \sigma_{\text{swap}}| \geq \tau \cdot \sigma_{\text{swap}}$) would conflate two unrelated things: slow organic LVR drift accumulated across many blocks, and single-block manipulation. After *any* genuine σ_q move, σ_{live} jumps immediately but σ_{swap} only catches up at $2^{-\text{EMA_SHIFT_BLOCKS}}$ per block, so the deviation stays above τ for $\approx 2^{\text{EMA_SHIFT_BLOCKS}}$ blocks and the gate stays *tripped that whole time*, locking out honest mints long after the move was over (on a realistic 12s jump tail this tripped honest mints $\sim 72\%$ of the time with a ~ 98 -minute worst-case lockout). The raw single-block signal separates the two: organic σ_q is flat between arbitrage jumps, so $|\sigma_{\text{live}} - \sigma_{\text{prevBlockEnd}}|$ is ≈ 0 in a quiet block regardless of how

far the pool has drifted over the day. The gate trips only *in* a block carrying a $\geq \tau_d$ jump and reopens the next — organic trip rate $\approx 3.8\%$ (volatile family) to $\approx 0.03\%$ (correlated-peg family), worst-case lockout 1–4 blocks. σ_{swap} is retained for the jobs its slow convergence is good at: the lagging b -anchor and the burn clamp.

3.4 Min-Clamped Burn and Why It Closes JIT

Burn is always available — the gate never disables it — but the *amount returned* is min-clamped. Concretely, for a requested burn fraction $\alpha = \text{lpAmount}/\text{lpSupply}$:

- **Full drain** ($\text{lpAmount} == \text{lpSupply}$): $\alpha' = \alpha = 1$, the burner receives the entire pool inventory. The min-clamp is bypassed because there is no remaining LP to receive the gated-out value.
- **Killed pool** (_killed is set): $\alpha' = \alpha$, pure proportional withdrawal. The min-clamp would unfairly penalize LPs caught with pending vig at kill time; the value would never reach honest LPs anyway since further pool activity is frozen.
- **Normal case**: $\alpha' = \alpha \cdot \min(\sigma_{\text{swap}}, \sigma_{\text{live}})/\sigma_{\text{live}}$. The user receives α' of every reserve token, while the LP-token burn is at the full α .

The pool inventory q scales by $(1 - \alpha')$ — the fraction actually paid out — while *both* the LP-token supply and σ_{swap} scale by $(1 - \alpha)$. Scaling σ_{swap} by the LP-supply factor $(1 - \alpha)$ rather than the reserve factor $(1 - \alpha')$ is deliberate, and the next paragraph shows why.

Why $(1 - \alpha)$: fragmentation invariance. With the clamp active ($\sigma_{\text{swap}} < \sigma_{\text{live}}$), a naive $(1 - \alpha')$ scaling of σ_{swap} would preserve the $\sigma_{\text{swap}}/\sigma_{\text{live}}$ ratio across the burn, but it would let a fast LP *fragment* their exit into many small burns and recapture part of the convergence-window penalty: each split chunk would face the same clamp ratio against a proportionally richer per-LP pool, so the sum of N small payouts would exceed the single-burn payout. Scaling σ_{swap} by $(1 - \alpha)$ instead holds the value-per-LP quantity

$$x = \frac{\sigma_{\text{swap}} \cdot R}{\sigma_{\text{live}} \cdot S}$$

invariant across the burn (R the reserve value, S the LP supply): the two numerator factors scale by $(1 - \alpha)$ and the two denominator factors by $(1 - \alpha')$, so every chunk in a fragmented exit — and any third party burning in the same block — sees exactly the x of the pre-burn pool. The aggregate of N split burns equals $\Delta L \cdot x = \alpha' \cdot R$, the single-burn payout. The mechanism is a *tightening* of the clamp on each subsequent same-block burn: because σ_{swap} shrinks more than σ_{live} , the next burner faces a stricter ratio, which is exactly the lever that keeps the missed vig with the slower-burning LPs. When $\alpha' = \alpha$ (no clamp, full drain, or killed) the $(1 - \alpha)$ and $(1 - \alpha')$ factors coincide and this reduces to plain proportional scaling.

Why JIT is closed structurally. Consider the attack: mint γ , observe an incoming swap that adds vig V to σ_q , burn α in the same block.

- Pre-attack: $\sigma_q = \sigma_{q,\text{pre}}$, $\sigma_{\text{swap}} = \sigma_{q,\text{pre}}$ (converged).
- Post-mint: $\sigma_{\text{swap}} = (1 + \gamma) \sigma_{q,\text{pre}} = \sigma_{\text{live}}$ (the mint scales both proportionally).
- Post-swap: $\sigma_{\text{live}} = (1 + \gamma) \sigma_{q,\text{pre}} + V$; σ_{swap} is unchanged (no mid-block EMA step).

- Burn $\alpha = \gamma / (1 + \gamma)$: $\alpha' = \alpha \cdot \sigma_{\text{swap}} / \sigma_{\text{live}}$; the payout is $\alpha' \cdot \sigma_{\text{live}} = \alpha \cdot \sigma_{\text{swap}} = \gamma \cdot \sigma_{q,\text{pre}}$.

The attacker receives back exactly their mint deposit, with no share of the vig V . The V stays in pool inventory and accrues to LPs who do *not* burn within the convergence window. No protocol mint fee is needed for this defense; honest LP entry incurs no protocol fee.

What honest LPs see. LPs that hold longer than roughly $4 \cdot 2^{\text{EMA_SHIFT_BLOCKS}}$ blocks ($\approx 3\text{--}14$ hours depending on shift) see $\alpha' \approx \alpha$. There is no exit penalty in the steady state. LPs forced to exit immediately after a vig-adding swap pay the min-clamp as an effective exit fee that decays over the EMA window, with the “missed” vig staying for slower-burning LPs.

3.5 Per-Window Rate Limit

The accumulator `_gammaAccum` decays continuously per block:

$$\text{_gammaAccum} *= (1 - 2^{-\text{EMA_SHIFT_BLOCKS}})^{\Delta}, \quad \Delta = \text{blocks since last update},$$

implemented as Δ linear shifts (each shift subtracts `_gammaAccum` $\gg$ `EMA\_SHIFT\_BLOCKS` from itself). This matches the σ_{swap} window scale, so the boundary-attack residual budget and the gate’s natural recovery share a single timescale.

Each mint or `swapMint`:

1. Decays `_gammaAccum` up to the current block.
2. Computes $\gamma_{\text{remaining}} = \text{MAX_GAMMA_PER_WINDOW_PPM} - \text{_gammaAccum}$.
3. Sets $\gamma_{\text{fill}} = \min(\gamma_{\text{request}}, \gamma_{\text{remaining}})$.
4. Reverts if $\gamma_{\text{fill}} \leq 0$ (no budget left).
5. Reverts with "rate limited" if $\gamma_{\text{fill}} < \gamma_{\text{request}}$ and the caller passed `partialFillAllowed = false`; otherwise proceeds with the partial fill.
6. After the mint, adds γ_{fill} to `_gammaAccum`.

Per-window rather than per-mint because a per-mint cap is cosmetic under transaction splitting; the per-window cap binds $\sum \gamma_i$ regardless of how many mints there are or who submits them. With JIT closed structurally by the min-clamped burn, `MAX_GAMMA_PER_WINDOW_PPM` is a backstop against attacker self-sandwich cycling rather than the primary JIT defense; the standard pools set it to 4,000 PPM (0.4% of supply per window) on the volatile OG pool and 10,000 PPM (1%) on the correlated-peg pool, which comfortably accommodates organic LP growth without re-opening the cycling surface.

3.6 Post-Mint LP Lock

The rate limit caps the *rate* of LP issuance, but a per-window cap alone is exhaustible at zero net cost: an attacker can atomically `mint` γ and `burn` it back in the same block, keeping the rate-limit credit (the accumulator already booked γ_{fill}) while the min-clamped burn returns the deposit at par (at the moment of attack $\sigma_{\text{swap}} = \sigma_{\text{live}}$, so $\alpha' = \alpha$). On an idle pool this collapses to a gas-only grieving DoS on honest minters. The post-mint LP lock closes this surface.

Mechanism. Every `mint` and `swapMint` appends a time-locked *cohort* to the receiver: the minted LP becomes non-burnable until block `block.number + MINT_LOCK_BLOCKS`. The lock is enforced in the ERC-20 `_update` hook — a *burn* that would dip into the receiver’s still-locked balance reverts, while a *transfer* of still-locked LP is permitted but carries the lock to the recipient (see *Cohort accounting* below), so the unlock block is preserved no matter who holds the tokens. The window `MINT_LOCK_BLOCKS` is a per-pool immutable, capped at 50,400 blocks (one week of L1 blocks) at the planner; the suggested L1 default is 300 blocks (≈ 1 hour), scaled up proportionally on shorter-block L2s to preserve the wall-clock deterrent. `initialMint` does not lock — it is operator-controlled and no attacker action precedes it.

Cohort accounting and transfers. Each account holds a FIFO list of cohorts, kept sorted by unlock block and lazily pruned as they expire; the list is bounded at `MAX_LOCK_ENTRIES = 32` live cohorts per account (a new mint to an account already at the cap reverts until a cohort expires). The lock travels with the tokens: a transfer that moves still-locked LP migrates the minimum FIFO prefix of cohorts to the recipient, each retaining its original unlock block, so the lock cannot be shed by forwarding LP to a fresh address. Subsequent mints never extend an existing cohort — each mint locks only its own freshly-issued LP.

Deterrence. Any non-zero window closes the same-block round-trip outright. Deterrence against *sustained* DoS scales with the window: the integrated locked capital an attacker must keep deposited to hold the rate limit pinned at $\Gamma_{\max}$ grows roughly linearly with the lock duration (at `EMA_SHIFT_BLOCKS = 4`, a one-hour lock pins on the order of $9\Gamma_{\max}$ worth of γ continuously deposited). Honest LPs that mint to hold are unaffected, as are aggregators and zappers that mint LP for an end user to keep — the lock only bites a position that tries to exit within the window.

3.7 Boundary Attack and the Swap-Fee Coupling

The gate caps an attacker’s per-bundle skew at $\delta_f \leq \tau \cdot \sigma_q$. Through the LMSR mint-sandwich mechanism, this lets them extract at most $\tau \cdot \gamma / (1 + \gamma) \cdot \sigma_q$ per attack at zero fee. With the pool’s per-asset swap fee applied to both the forward and reverse skew legs, the per-attack net flips negative: the swap fees on the two legs exceed the gate-bounded gross extraction at the per-family (τ, f) recommendations in `doc/rate-limited-mints.md` (and verified by the boundary-attack simulator). The boundary attack is therefore *closed* (net-negative for the attacker), not merely bounded.

3.8 Synchronous Semantics and the Optional Mint Queue

`mint` and `swapMint` are synchronous at the pool: the user submits one transaction and either receives LP in the same transaction, sees a partial fill (if they opted in), or sees the call revert with one of the operational reverts ("`volatile market`", "`rate limited`", "`slippage control`", "`insufficient funds`"). The pool holds no queue and runs no deferred execution itself. Because the rate limit can force a partial fill, the `PartyConcierge` router offers an *optional* mint queue with permissionless keeper execution as a convenience (§8.4); a user who opts out sees the plain synchronous semantics above.

3.9 Attack Surfaces Closed and Residual Surfaces

Closed.

1. Mint sandwich (gate caps skew + swap fee makes the per-attack residual net-negative).
2. Spam-fill EMA bypass (block-anchored step ignores intra-block transaction count).
3. Quiet-pool single-shot convergence (fixed one-shift-per-block step regardless of elapsed blocks).
4. JIT mint (min-clamped burn structurally returns the attacker’s deposit without any vig share).
5. Burn-side sandwich (verified arithmetically self-griefing for any attacker LP share $\lambda_a \leq 1 - \alpha$, see `doc/rate-limited-mints.md` “Burn safety”).
6. Atomic `mint`→`burn` rate-limit exhaustion (the post-mint LP lock forces every mint to commit capital across blocks before it can exit, §3.6).
7. Burn fragmentation (the $(1 - \alpha)$ scaling of σ_{swap} makes a clamped exit fragmentation-invariant, so splitting a burn cannot recapture the convergence-window penalty, §3.4).

Residual.

1. **Sustained DOS griefing.** An adversary willing to burn capital indefinitely can keep the gate tripped by continuously re-skewing. Per the threat model this is unprofitable extraction for the attacker; honest mints simply revert and retry. No tokens get stuck.
2. **LP-as-oracle.** The gate controls mints; pair prices can still swing within $\exp(\tau)$ under a gate-permitted in-bounds skew. At the standard-pool threshold $\tau = 30$ PPM (`MINT_DEVIATION_PPM` on both the OG and correlated-peg pools) this is approximately $1.00003\times$, negligible. Downstream consumers that read pool marginal price for liquidations should still apply their own TWAP.
3. **Initial-mint window.** The very first mint occurs in the same transaction as `initialMint`, which is operator-controlled; no attacker action is possible before initial mint.
4. **Rate-limit DOS.** An attacker can exhaust `MAX_GAMMA_PER_WINDOW_PPM` by minting their own LP to grief honest users. The post-mint LP lock (§3.6) removes the zero-cost atomic version of this: held pressure on the rate limit now requires capital locked for `MINT_LOCK_BLOCKS`, with the integrated locked capital growing roughly linearly in the lock window. What remains is a capital-bound DOS, not a value-extraction surface.

4 Liquidity Operations

The pool issues an ERC-20 LP share token. The total LP supply L tracks $S(\mathbf{q})$ proportionally: the deployer specifies the initial supply $L^{(0)}$ as an argument to `initialMint` (defaulting to 10^{18} if zero is passed), independent of pool size, and subsequent operations preserve the ratio $L/S(\mathbf{q})$ on every proportional change. The LP-share ERC-20 reports `decimals() = 18` regardless of $L^{(0)}$; the deployer typically chooses $L^{(0)}$ to land the pool at a convenient initial share price relative to the initial deposit basket.

4.1 Proportional Mint

The `mint` entry point is gated by the wedge defense (§3) — the raw $\Delta\sigma_q$ gate and the per-window rate limit — and partial-fill aware. Its full signature is

```
mint(payer, fundingSelector, receiver, lpTokenAmount, maxAmountsIn, minLpOut,
    partialFillAllowed, deadline, cbData) → (lpMinted, gammaFilled).
```

The caller specifies a target LP amount $\Delta L = \text{lpTokenAmount}$. The pool:

1. Captures the gate reference $\sigma_{\text{prevBlockEnd}}$ and steps σ_{swap} if this is the first state change of a new block (§3.2).
2. Decays the per-window accumulator `_gammaAccum` (§3.5).
3. Checks the raw $\Delta\sigma_q$ gate, comparing the pre-mint σ_{live} against the block-start reference $\sigma_{\text{prevBlockEnd}}$ (§3.3); reverts "volatile market" if tripped.
4. Computes the requested $\gamma_{\text{request}} = \Delta L / L$, clamps to $\gamma_{\text{fill}} = \min(\gamma_{\text{request}}, \gamma_{\text{remaining}})$, reverts "rate limited" if $\gamma_{\text{fill}} \leq 0$ or if $\gamma_{\text{fill}} < \gamma_{\text{request}}$ and the caller did not pass `partialFillAllowed = true`.
5. For each token j , computes the deposit needed as $\gamma_{\text{fill}} \cdot q_j$, reverts "slippage control" if any deposit exceeds the per-token `maxAmountsIn[j]` bound.
6. Pulls funds via the chosen funding mode (§7). Any over-delivery from a funding callback is absorbed into reserves (donated to LPs) per the rounding policy (§6.4).
7. Applies the proportional mint: $q_j \mapsto (1 + \gamma_{\text{fill}}) q_j$ for each j , and mints $\gamma_{\text{fill}} \cdot L$ LP shares to `receiver`. Reverts "slippage control" if `lpMinted < minLpOut`.
8. Scales σ_{swap} and the gate reference $\sigma_{\text{prevBlockEnd}}$ proportionally (both $\ast = (1 + \gamma_{\text{fill}})$, §3.2) and credits the accumulator (`_gammaAccum += \gamma_{\text{fill}}`).
9. Appends a post-mint lock cohort on the minted LP at `receiver`, non-burnable for `MINT_LOCK_BLOCKS` (the LP stays transferable; the lock migrates with it, §3.6).
10. Returns $(\text{lpMinted}, \text{gammaFilled})$. For typical mints inside the window budget, $\text{lpMinted} = \Delta L$.

There is **no protocol mint fee**; honest LP entry is free. The JIT defense lives entirely on the burn side (§3.4).

4.2 Proportional Burn

`burn(payer, receiver, lpAmount, minAmountsOut, deadline, unwrap)` burns $\Delta L = \text{lpAmount}$ LP shares and returns the per-token withdraw amounts. The full payout rule, including the JIT-defense min-clamp, is specified in §3.4; the summary is:

- The pool computes the requested burn fraction $\alpha = \Delta L / L$ and the effective payout fraction α' per §3.4 (with the full-drain and killed-pool edge cases bypassing the min-clamp).
- The pool sends $\alpha' \cdot q_k \cdot \beta_k$ uint of each token k to `receiver`; if `unwrap` is set and token k is the configured native wrapper, the output is unwrapped to native currency before delivery.

- Per-token `minAmountsOut[k]` bounds are enforced; revert string "slippage control".
- The reserves $\mathbf{q}$ scale by $(1 - \alpha')$ (the amount paid out), while the LP-token supply and σ_{swap} both scale by $(1 - \alpha)$. The $(1 - \alpha)$ scaling of σ_{swap} makes a clamped burn fragmentation-invariant (§3.4).

`burn` is *not* gated by the deviation check and is *not* disabled by `kill()`; LPs can always exit pro rata.

Proportional mint and burn are value-neutral in the fee-free kernel at matched b : every pairwise marginal price is invariant under $\mathbf{q} \mapsto \lambda \mathbf{q}$ when the corresponding b also scales by λ .

4.3 Single-Asset Mint: `swapMint`

`swapMint` is exact-LP-out: the caller specifies a target ΔL and a `maxAmountIn` cap on the deposit of a single input asset i . The operation is equivalent to executing the externally observable sequence “swap $i \rightarrow j$ for every $j \neq i$, then proportional mint with the resulting basket.” The fee-free kernel computes the required input in one forward pass against a memory-only local state $\mathbf{q}_{\text{local}}$, with b anchored to the same block-aligned $\kappa \cdot \min(\sigma_{\text{swap}}, \sigma_{\text{live}})$ used by the `swap` kernel (§2.4):

1. Compute $\gamma = \Delta L / L$ and $\beta = \gamma / (1 + \gamma) \in (0, 1)$.
2. Initialize $\mathbf{q}_{\text{local}} = \mathbf{q}$ and anchor $b = \kappa \cdot \min(\sigma_{\text{swap}}, \sigma_{\text{live}})$ once for the entire chain.
3. For each $j \neq i$ in fixed index order, evaluate $r_{0,j} = \exp((q_{\text{local}}^j - q_{\text{local}}^i) / b)$ and solve the Hanson exact-out inverse for the cost of acquiring βq_j of asset j :

$$x_j = b \ln \left(\frac{r_{0,j}}{r_{0,j} + 1 - e^{\beta q_j / b}} \right).$$

Update $\mathbf{q}_{\text{local}}$: $q_{\text{local}}^i += x_j$, $q_{\text{local}}^j -= \beta q_j$.

4. Return the total deposit $a = \frac{\beta q_i + \sum_{j \neq i} x_j}{1 - \beta}$.

The chain mutates a memory copy only; the pool’s storage is updated at the end with the new reserves and the minted LP shares. Crucially, b *does not re-evaluate per leg*: every leg of the multi-step chain prices against the same block-aligned b as the standalone `swap` kernel, collapsing the three operations (`swap`, `swapMint`, `burnSwap`) onto a single intra-block pricing surface and closing the cycle-extraction surface that per-leg re-anchoring would otherwise open (§2.4, “Why fixed- b within a block”).

End-to-end flow. The pool runs the raw $\Delta \sigma_q$ gate, the rate-limit decay, and the funding-mode pull around the kernel call, with the same partial-fill and revert semantics as `mint`:

1. Step σ_{swap} and capture $\sigma_{\text{prevBlockEnd}}$ if first-of-block; absorb any retained fee backlog into the gate reference (§3.2); decay `_gammaAccum`.
2. Pull the input token via the chosen funding mode; apply the full swap-leg fee — the input asset’s fee *plus* the other assets’ fees, the same combined rate a decomposed multi-leg `swap` would charge (§6) — and send the fee-discounted amount through the kernel.

3. Apply the rate-limit cap to γ_{request} ; revert "rate limited" if $\gamma_{\text{fill}} < \gamma_{\text{request}}$ and `partialFillAllowed = false`.
4. Check the raw $\Delta\sigma_q$ gate against the post-swap-leg σ_{live} relative to $\sigma_{\text{prevBlockEnd}}$ — the non-strict $\geq$ catches the *poison-swapMint at the gate boundary* described in §3.3; revert "volatile market" if tripped.
5. Apply the proportional mint; update σ_{swap} and `_gammaAccum`; append a post-mint lock cohort on the minted LP (§3.6); revert "slippage control" if `maxAmountIn` or `minLpOut` fails; emit `SwapMint`.

Failure of any step reverts the entire operation; tokens remain with the caller (or never leave them, in the Permit2 and callback cases).

Rounding. The exact-out cost x_j uses ABDK floor primitives that round in the trader's favor under the natural `mul/div/exp/ln` semantics. To restore the LP-favoring invariant, the kernel applies explicit ceiling helpers (`_ceilMul`, `_ceilDiv`, `_ceilExp`, `_ceilLn`) at every step whose floor direction would shave a sub-ulp gift to the swapper, and adds a 1-ulp cushion in the asymptotic `EXP_LIMIT` branch. Every individual rounding decision is documented in-source by signed partial derivative against the LP-favoring direction.

4.4 Single-Asset Redeem: `burnSwap`

`burnSwap` is the inverse of `swapMint`: the caller burns ΔL LP shares and receives a single output asset j . The operation is *not* subject to the $\Delta\sigma_q$ gate or the rate limit — both apply to mint-side pool expansion, not to withdrawal — but the proportional-burn portion of the payout is subject to the same min-clamp as standalone `burn` (§3.4). The fee-free kernel computes the payout by simulating a proportional withdraw followed by swaps of every withdrawn a_k back into asset j against the already-burnt local state, with b anchored to the block-aligned $\kappa \cdot \min(\sigma_{\text{swap}}, \sigma_{\text{live}})$ for every leg:

1. Compute $\alpha = \Delta L / L$ and the effective $\alpha' = \alpha \cdot \min(\sigma_{\text{swap}}, \sigma_{\text{live}}) / \sigma_{\text{live}}$ (with edge cases per §3.4).
2. Anchor $b = \kappa \cdot \min(\sigma_{\text{swap}}, \sigma_{\text{live}})$ once for the entire chain. Initialize $\mathbf{q}_{\text{local}} = (1 - \alpha') \mathbf{q}$ and start the payout with the proportional share $Y_j = \alpha' q_j$.
3. For each $k \neq j$: set $a_k = \alpha' q_k$ (the withdrawn portion of asset k). Evaluate $r_{0,k} = \exp((q_{\text{local}}^j - q_{\text{local}}^k) / b)$ and the Hanson exact-in payout

$$y_{k \rightarrow j} = b \ln(1 + r_{0,k} (1 - e^{-a_k/b})).$$

Accumulate $Y_j += y_{k \rightarrow j}$ and update $\mathbf{q}_{\text{local}}$. If $y_{k \rightarrow j}$ exceeds available q_{local}^j , cap the payout to q_{local}^j and derive the actually-consumed input via the exact-out inverse.

4. Per-asset numerical degeneracy (non-positive inner `ln` argument, `EXP_LIMIT` overshoot) is handled by the asymptotic branch in `LMSRKernel.swapAmountsForBurn` described in §2, contributing the bounded asymptotic payout rather than aborting the redeem.

As with `swapMint`, b does not re-evaluate per leg. The standalone `swap`, `swapMint`, and `burnSwap` operations all see the same block-aligned b , so intra-block closed cycles compose against one consistent pricing surface. The full swap-leg fee — the output asset's fee *plus* the other assets' fees — is applied to the gross output (§6).

Kill-resistance asymmetry. `burnSwap` is disabled by `kill()`; `burn` is not. A killed pool therefore permits proportional redemption but not single-asset redemption. `burnSwap` also pays the full swap-leg fee on its swap-back leg; the proportional `burn` does not.

5 LP Protection Mechanisms

The pricing kernel (§2) and liquidity operations (§4) carry per-primitive rounding and approximation choices that resolve every sub-ulp ambiguity in the LP’s favor. This section consolidates those guarantees and adds the admin-side LP protections that constrain the operator surface. Each mechanism is enforced in source; this is an index, not a statement of intent.

5.1 Kernel-Level LP Safety

Block-anchored b never overshoots LS-LMSR. The production swap kernel (`LMSRKernel.swapAmountsForExactInput`) evaluates the single-pass Hanson exact-input formula at $b = \kappa \cdot \min(\sigma_{\text{swap}}, \sigma_{\text{live}})$ (§2.4, §2.6). The output paid to the swapper is bounded above by the output the path-exact LS-LMSR mapping would pay under the same inputs: the block-fixed b is at most the path-exact b -trajectory, and the Hanson exact-in mapping is monotone increasing in b , so the kernel’s output is at or below the path-exact reference. The swapper *never* receives more than the path-exact LS-LMSR; the LP *never* loses more than the path-exact LS-LMSR.

Single intra-block pricing surface. Standalone `swap`, the per-leg sub-swaps inside `swapMint`, and the per-leg sub-swaps inside `burnSwap` all evaluate at the same block-aligned b . Closed intra-block cycles combining the three operations therefore price against one consistent surface and cannot extract value across pricing inconsistencies. The deployed test suites `test/SwapMintBurnSwapBAnchorExploit.t.sol`, `ArbHarness`, `StandardPoolAttacks`, and `PoC_WedgeAttack` verify the strict-zero-extraction property across the recommended operating envelope.

Capacity caps. Output is hard-capped at available inventory $y \leq q_j$ in the kernel. The cap-and-invert branch derives the input that the LP actually absorbs, so the pool is never required to over-pay relative to its inventory.

Single-asset chain LP-favor rounding. The `swapMint` and `burnSwap` chains (§4.3–§4.4) carry a per-primitive rounding discipline. Where ABDK’s default `mul/div/exp/ln` floor would shave a sub-ulp gift to the swapper, the kernel substitutes a ceiling helper (`_ceilMul`, `_ceilDiv`, `_ceilExp`, `_ceilLn`); where the floor direction already serves LP-favor, the default is kept. Every individual decision is documented in-source by signed partial derivative against the LP-favoring direction. The `EXP_LIMIT` branch in `swapMint` carries a 1-ulp cushion on top of the ceiling chain so that future ABDK-level rounding changes cannot re-open a leak.

5.2 Fee and Protocol-Share Rounding

- **Total fees round up** for the pool and against the taker.
- **Protocol share rounds down** for the LP and against the protocol developers.
- **Outputs to users floor; inputs from users ceil.**

- **Pre-funded dust** is credited to LP reserves rather than refunded to the caller.

The protocol developers absorb the rounding loss on the protocol-share split; the LP keeps the residual. This is deliberate. Section 6 specifies the fee mathematics this rounding policy applies to.

5.3 Correlated-Asset LVR Reduction (Conditional)

A structural LP benefit of the multi-asset LMSR geometry is a closed-form *cross-correlation discount* on the loss-versus-rebalancing (LVR) rate (Olson & Claude Opus, 2026, in-tree doc/lvr-multiasset-lmsr.pdf): the more correlated the pool’s basket, the more the LMSR’s shared-simplex coupling cancels the relative-mode arbitrage that bleeds value from separable bilateral CFMMs. For perfectly correlated equal-volatility baskets the relative-mode contribution vanishes entirely.

This protection is *conditional*: it is realized through pool composition, not enforced by the protocol on every pool, and a pool of uncorrelated assets does not receive it. The operator-side mechanics — basket-selection criteria, the discount formula, and quantitative comparison of candidate compositions — are in §10.2.

5.4 Admin-Side LP Protection

The admin (planner owner / pool owner) cannot reach LP reserves. The exhaustive CAN/CANNOT inventory is in doc/security/admin-powers.md; the LP-relevant summary is:

- Admin **cannot** mint LP out of band, burn user LP, change κ , change fees on a live pool, pause user-side **burn**, blacklist or freeze user balances, upgrade the pool implementation, renounce ownership, or move funds out of the pool.
- Admin **can** deploy new pools, set the protocol-fee recipient on a live pool, and invoke the irreversible `kill()` switch on a pool they own.

Compromised-key worst case. A leaked admin key cannot reach LP reserves. Worst case: the attacker calls `kill()` (disabling new deposits, swaps, and single-asset withdrawals) and rotates `protocolFeeAddress` so any not-yet-collected protocol fees route to the attacker on the next sweep. Both **burn** (proportional LP redemption) and `collectProtocolFees` (protocol-fee sweep) remain operational on a killed pool, so LPs can always withdraw their pro-rata share of pool reserves and any party may push already-accrued protocol fees to the configured `protocolFeeAddress`. The core team retains the ability to `kill()` the system as a final defensive measure — even if the admin key is compromised, the system can be put into the LP-can-only-exit state. **A leaked admin key presents little to no danger to LP reserves**; it is a denial-of-service surface and a partial protocol-fee redirect surface, not a fund-loss surface.

The admin key is held on a hardware wallet (see §13).

5.5 Burn Is Always Live

burn (the proportional-redemption path) is not gated by `kill()`. The proportional-redemption guarantee is preserved across every kill scenario, every admin-key scenario, and every operator-misconfiguration scenario by construction: there is no code path in the protocol that can disable **burn** for an LP holding the LP token. `collectProtocolFees` is likewise un-killable, so

already-accrued protocol fees are also recoverable on a killed pool — they sweep to the configured `protocolFeeAddress` regardless of kill state. These are the only two state-mutating entry points that remain live after `kill()`; every other mutating call reverts.

The one restriction on `burn` is the post-mint LP lock (§3.6): LP is non-burnable for `MINT_LOCK_BLOCKS` after it is minted. This is a temporary, self-incurred, time-bounded restriction on freshly-minted shares — not an admin lever and not a kill-state effect — and it expires automatically. Any LP held past its lock window, including all LP on a killed pool once the window elapses, redeems pro rata without qualification.

6 Fee Structure

Direction summary. Each user-facing entry point charges its fee on exactly one side of the operation. `swap` and `burnSwap` are **fee-on-output**: the fee-free kernel computes the gross payout first, then the fee is deducted from that payout before the user receives it. `swapMint` is **fee-on-input**: the fee is deducted from the user-submitted input before the fee-free kernel runs, so the kernel prices the net (post-fee) input. The proportional `mint` and proportional `burn` entry points are **fee-free**: no swap fee applies to either side, and JIT extraction is closed structurally by the min-clamped burn (§3.4) rather than by a fee. The collected fee is retained in the pool, in the token on the fee’d side, and split between the protocol share (§6.3) and the LP share (kept in reserves, raising $S(\mathbf{q})$ and the LP-share-denominated value).

6.1 Per-Asset Swap Fees

Each asset k has an immutable fee rate f_k set at deployment, expressed in parts-per-million. The deployer enforces $f_k < 10,000$ ppm ($< 1\%$). For an asset-to-asset `swap` $i \rightarrow j$, the effective pair fee is the additive composition

$$f_{\text{eff}}(i, j) = f_i + f_j, \quad (11)$$

applied to the gross kernel output on the output side:

$$y_{\text{net}} = y_{\text{gross}} (1 - f_{\text{eff}}).$$

The full user-submitted input a_{user} is forwarded to the fee-free kernel, which returns the gross output y_{gross} ; the user receives exactly y_{net} in token j , and the pool retains $y_{\text{gross}} - y_{\text{net}}$ in token j as the collected fee (then split into protocol/LP shares).

The additive form is preferred over the exact multiplicative $1 - (1 - f_i)(1 - f_j)$ because the cross-term at the deployment cap ($f_i, f_j < 10^{-2}$) is below 10^{-4} of the pair fee, and the simpler formula is cheaper to evaluate and easier for users to reason about.

6.2 Compound-Operation Fees on `swapMint` and `burnSwap`

`swapMint` and `burnSwap` each carry a swap leg (the named asset is traded against all the others) and a fee-free proportional leg. The swap leg is charged the *full* swap-leg fee, not just the named asset’s own rate. Writing f_m for the named asset’s fee (the input i on `swapMint`, the output j on `burnSwap`) and n for the basket size, the combined rate is

$$f_{\text{leg}} = f_m + \left\lceil \frac{1}{n-1} \sum_{k \neq m} f_k \right\rceil, \quad (12)$$

the named asset’s fee plus the (ceiling-rounded) average of the other assets’ fees. This matches what the equivalent decomposed multi-leg `swap` path — the named asset traded into each of the

$n - 1$ others — would charge under the additive pair-fee composition $f_i + f_j$ of §6. For a two-asset pool it reduces exactly to $f_i + f_j$. The fee *side* matches the side on which the user moves tokens:

- **swapMint** is **fee-on-input**: the pool computes $a_{\text{eff}} = a_{\text{user}}(1 - f_{\text{leg}})$ and passes a_{eff} to the fee-free kernel, which mints exactly ΔL LP shares (exact-LP-out) for the gate-and-rate-limited fraction γ_{fill} . The fee $a_{\text{user}} - a_{\text{eff}}$ is collected in the deposited token.
- **burnSwap** is **fee-on-output**: the fee-free kernel computes the gross payout y_{gross} for the burned LP fraction (post the min-clamp scaling, §3.4); the user receives $y_{\text{net}} = y_{\text{gross}}(1 - f_{\text{leg}})$. The fee $y_{\text{gross}} - y_{\text{net}}$ is retained in the withdrawn token.

In both cases the protocol share is split off and the remainder stays in pool reserves. There is no separate LP-token-side fee on either operation, and no protocol fee on the proportional **mint** or **burn**. Charging the *full* swap-leg fee — rather than only the named asset’s rate — closes a fee-bypass loophole: otherwise an attacker could pair a compound operation with the fee-free proportional **burn** or **mint** and pay only one side’s per-asset fee.

6.3 Protocol Fees

A protocol-fee share ϕ (ppm) is taken from every collected fee. The deployer enforces $\phi < 300,000$ ppm ($< 30\%$). Accrued protocol fees are tracked in a per-token ledger (`_protocolFeesOwed`) excluded from the pool’s liquidity-bearing balance (`_cachedUintBalances`); pending protocol fees do not participate in pricing or earn further fees. The `collectProtocolFees()` entry point sweeps the ledger to the configured `protocolFeeAddress` and zeroes it. The recipient is read at call time, so a recipient rotation applied between accrual and collection retroactively redirects pending fees (Uniswap V3-style semantics); operators that need atomic handoff must collect before rotating.

Public, unguarded sweep. `collectProtocolFees()` is intentionally *permissionless* and carries no signature, owner-gate, or other authorization check beyond the reentrancy guard. Funds always sweep to the admin-set `protocolFeeAddress` regardless of which address called the function. This is a deliberate operational simplification: the sweep can be triggered by any keeper, integrator, or monitoring bot without coordinating a signature from the operator’s hardware-wallet key, while the destination remains under admin control via the standard `setProtocolFeeAddress` surface. The only address that benefits from the sweep is the address the admin chose; no caller can divert proceeds.

6.4 Rounding Policy

All rounding decisions are biased to protect existing LP value:

- Total fees are rounded *up* (taker pays the extra ulp; LP keeps it).
- The protocol share is rounded *down* (LP keeps the residual; the protocol developers absorb the ulp).
- Pool outputs to users are floored; pool inputs from users are ceiled.
- Pre-funded amounts above the exact required amount are credited to reserves rather than refunded (LP gift).

- Q64.64 ceil helpers (`_ceilMul`, `_ceilDiv`, `_ceilExp`, `_ceilLn`) are applied per-primitive in `swapMint`/`burnSwap` to restore LP favor where ABDK's default floor would otherwise leak a sub-ulp gift to the swapper.

7 Funding Modes

The user-facing entry points that pull external tokens into the pool — `swap`, `mint`, and `swapMint` — accept any of four token-delivery modes via a `fundingSelector` parameter. The kernel mathematics are identical across modes.

No funding mode on burn paths. `burn` and `burnSwap` do *not* take a `fundingSelector` and do not pull external tokens. The asset the caller spends on these operations is the LP token itself, and the pool *is* the LP token's issuing contract — the LP-token debit is a storage write in the same contract that holds the pool reserves, so no ERC-20 `transferFrom`, callback, or Permit2 signature is required to move value into the pool on a burn. The caller's LP balance (or an existing LP-token allowance) is the only authorization surface. This is why none of the four funding modes below applies to either burn entry point.

APPROVAL Standard ERC-20 `transferFrom` from the caller. Caller must hold the input balance and have approved the pool. The mode requires `msg.sender == payer` to prevent third-party-allowance abuse.

PREFUNDING Caller deposits tokens to the pool address before invoking the entry point; the pool measures the balance delta against `_cachedUintBalances` + `_protocolFeesOwed`. Excess above the required amount is gifted to LPs. Native ETH is auto-wrapped via the configured wrapper when the pool's input token equals the wrapper address and the pool's ETH balance covers the amount. This delta is *unauthenticated*: it carries no depositor identity, and the `msg.sender == payer` gate only prevents abuse of a victim's *allowance*, not consumption of pre-deposited tokens. PREFUNDING is therefore intended only for integrating contracts that bundle the transfer and the entry-point call *atomically in one transaction*; a non-atomic deposit (transfer in one tx, call in another) can be front-run and consumed by any caller who sets `payer = msg.sender`. This residual race is a deliberate design choice, not a defended invariant — EOA and cross-transaction integrations should use APPROVAL, PERMIT2, or callback funding instead.

CALLBACK Aggregators and routers implement a callback interface. The pool calls `fundingSelector(nonce, token, amount, cbData)` on the `payer` address and measures the resulting balance delta to determine the funded amount.

PERMIT2 Uniswap Permit2 signature-based transfer with witness data binding the operation's parameters to the signature. Witness types are defined per entry point in `PartyPoolPermit2Witness` for `swap`, `mint`, and `swapMint`.

Native ETH. The pool accepts native ETH on the entry points declared `payable`, but only the configured wrapper (a WETH-style contract) may push raw ETH back into the pool via `receive()`. Any other direct ETH transfer reverts so stray sends cannot be stranded. Multi-asset mint loops carry an explicit per-call ETH budget so the wrap branch cannot over-claim against a constant `msg.value`.

8 Contract Architecture

8.1 PartyPool: Proxy and Storage

`PartyPool` is a non-upgradeable `DELEGATECALL` proxy. Two implementation libraries (`PartyPoolExtraImpl1`, `PartyPoolExtraImpl2`) host the body of `initialMint`, `mint`, `burn`, `swapMint`, `burnSwap`, `collectProtocolFees`, and the deployment-time `init` routine. The pool’s own runtime is a thin facade so its deployed bytecode plus its creation-code embed in `PartyPoolInitCode` both fit under EIP-170’s 24 KB contract-size cap. Implementation addresses are immutable; there is no upgrade path.

Storage layout is mirrored by `PartyPoolStorage.PoolState` for use under `DELEGATECALL`. Per-token `bases` and `fees` are not held in storage — they live in an `SSTORE2` “BFStore” data contract deployed at `init` time, whose address is captured in the `IMMUTABLE_BFSTORE` pool immutable. Hot paths read it via `EXTCODECOPY`, avoiding bounds-check `SLOADs` on the array length.

The LMSR state (`_lmsr.kappa`, `_lmsr.qInternal`) is a cache: the pool re-derives the post-swap, post-mint, and post-burn `qInternal` from the cached uint balances and the immutable bases on every mutation, keeping the kernel state synchronized with physical reserves across all state-changing paths.

8.2 PartyPlanner: Factory and Registry

`PartyPlanner.newPool` (`onlyOwner`) deploys a new `PartyPool` via `CREATE2` with operator-supplied parameters (see §9). The planner indexes deployed pools and tokens for off-chain discovery (`getAllPools`, `getPoolsByToken`, `tokenIndex`). A delta-equality check on the initial deposit (`balanceAfter - balanceBefore == initialDeposits[i]`) at deploy time rejects fee-on-transfer and rebasing tokens that fail to deliver the exact deposit amount. Pre-existing balances at the `CREATE2` address are absorbed into reserves and gifted to the first depositor; this is intentional and closes a deployment-griefing DoS that total-equality would have re-opened (see `doc/security/threat-model.md §J.6`).

8.3 PartyInfo: View Helpers

`PartyInfo` is a singleton view contract that accepts a target pool address. It hosts pricing utilities (`price`, `poolPrice`, `swapAmountsForExactPrice`, quote helpers for the multi-step mint/burn chains), the BFStore decoders (`denominators`, `fees`), and a `working(pool)` health check. Quote helpers run the same simulation as the on-chain entry points; their output is suitable for direct submission to `swap/swapMint/burnSwap`.

8.4 PartyConcierge: Router

`PartyConcierge` is the user-facing router. It accepts token addresses rather than indices so wallets can render trades with human-readable token names (EIP-7730 clear-signing compatibility). A single ERC-20 approval to the Concierge covers every pool on the same chain. Funding is callback-based: the Concierge stores call context in EIP-1153 transient storage (`cbUser`, `cbPool`, `cbMode`, `cbEthBudget`), validates `msg.sender == cbPool` on every callback, and refunds residual ETH via `sweepEth`. Adds approximately 10k gas to a typical swap.

The Concierge supports three input paths: standard approval pull, Permit2 signature-based pull (no Concierge approval required, only the universal Permit2 approval), and native-ETH wrap from `msg.value`.

8.4.1 Mint Queue and Keeper Mechanism

Because the pool’s per-window rate limit (§3.5) can force a `mint` or `swapMint` to fill only partially — or to revert outright when the window budget is momentarily exhausted, the gate is transiently tripped, or the market is moving — the Concierge offers an *optional* queue so a user can submit once and have the remainder filled automatically as budget frees up in later blocks. The queue is a convenience layer on top of the synchronous pool entry points; it is opt-in (a `useQueue` flag) and requires the caller to allow partial fills.

Try-first, then enqueue. A queued request first attempts a synchronous fill in the submitting transaction. If it fully fills, nothing is queued. If it partially fills, or reverts with one of the recoverable operational reverts (rate limited, volatile market, slippage, too-small, lock-list-full), the unfilled remainder is appended to a per-pool FIFO queue; non-recoverable reverts (deadline, killed pool, and the like) bubble up unchanged and queue nothing.

No cross-block token escrow. Consistent with the protocol-wide rule that the system never custodies a user’s tokens across blocks, a queued request holds *no* ERC-20 tokens. The queue entry records the requester, recipient, target pool, remaining LP to mint, a single *output tolerance* τ_s (`tolerancePpm`), and a deadline; the tokens stay in the requester’s wallet under their existing approval and are pulled only at the moment a keeper executes a fill. Crucially the entry stores a *tolerance*, not fixed per-token `maxAmountsIn` caps: because a proportional basket mint is price-neutral, the keeper re-derives the exact proportional draw caps from *live* reserves each block and reserves the slippage intent on the LP-output floor, $minLpOut = lpFill \cdot (1 - \tau_s)$. A request whose owner has since moved or de-approved the tokens (or whose live fill would breach τ_s) simply fails its execution attempt and is requeued or, past its deadline / timeout, dropped — no funds can be stranded in the router.

Queue funding: allowance or Permit2. A queued request may be funded two ways, selected by a per-request flag. (i) A standing ERC-20 allowance to the Concierge: each keeper tranche pulls its draw under the user’s existing approval. (ii) A standing Permit2 `AllowanceTransfer` allowance: dedicated `mintWithQueuePermit2Allowance` and `swapMintWithQueuePermit2Allowance` entry points register the allowance once at enqueue time (the user signs a single Permit2 `PermitBatch` / `PermitSingle`), and each subsequent keeper tranche draws against it via `transferFrom` with no further signature. Either way the funds never leave the requester’s wallet until a tranche actually executes.

Fragmentation into rate-limit tranches. A single large queued request is filled across many blocks in *tranches*, never as one all-or-nothing fill. On each `executeMints` call the keeper sizes the tranche to the pool’s *current* per-block γ -window budget (§3.5): it replicates the pool’s γ -accumulator decay off-chain to find the largest LP amount that fits the remaining budget this block, draws exactly that, and decrements the request’s remaining LP. The keeper recomputes the proportional caps and the *minLpOut* floor from the stored tolerance τ_s against live reserves for *every* tranche, so each piece independently honors the user’s slippage tolerance and a request that spans a price move is never filled at a stale ratio. Successive tranches drain the request as budget reopens, until it fully fills, hits its deadline, or persistently fails slippage for `SLIPPAGE_TIMEOUT_BLOCKS` (default 300) and is canceled.

Permissionless keepers and the two-part fee. Any address may call `executeMints(pool, maxCount)` to drive queued requests from the head, executing each as a tranche against the live pool until the window budget is spent. Keepers are compensated by two immutable Concierge-level fees, set at the router’s construction and applied uniformly across every pool it serves:

- A small fixed **native keeper fee** (`NATIVE_KEEPER_FEE`), denominated in the chain’s native asset and sized to cover a keeper’s execution gas in the worst supported case (a proportional mint into a large basket) plus a slim premium. The submitting user attaches exactly this amount as `msg.value`; it is held as native escrow (the only value the router ever holds across blocks) and paid to the keeper that drives the request to a terminal state. A request that fully fills synchronously refunds it immediately.
- A **percentage keeper fee** of 0.10% (`KEEPER_FEE_PPM = 1000 ppm`), skimmed from each token the fill actually consumes and paid to the executing keeper. Its headroom is reserved by shrinking the tranche’s LP target so the pool draw plus the post-draw skim always fits the requester’s balance and allowance — the FIFO head can never be wedged by a request that cannot cover the skim. This is the ongoing incentive that makes draining the queue profitable for keepers as soon as rate-limit budget reopens.

Cancellation and expiry. A requester may cancel their own queued request. If the target pool has been killed, the native escrow is refunded to them directly; on a live pool the entry is replaced by a tombstone and the escrow is forfeited to the next keeper that sweeps it (an anti-griefing measure, so a flood of submit-then-cancel requests cannot make keepers work for free). Keeper-side termination on an expired deadline or a stale, persistently-unfillable request likewise pays the native escrow to the terminating keeper. Queued mints carry no keeper or queue machinery on native-asset baskets, which are excluded from the queue because keeper execution is non-payable and cannot auto-wrap ETH.

9 Admin Powers and Parameter Fixity

The deployment parameter tuple

$$(\kappa, \mathbf{f} = (f_0, \dots, f_{n-1}), \phi, \tau, s, \Gamma_{\max}, \ell, \beta = (\beta_0, \dots, \beta_{n-1}))$$

is captured at `newPool` and is immutable for the lifetime of the pool. The wedge-defense parameters $(\tau, s, \Gamma_{\max})$ correspond to `MINT_DEVIATION_PPM`, `EMA_SHIFT_BLOCKS`, and `MAX_GAMMA_PER_WINDOW_PPM` respectively, and ℓ to the post-mint LP-lock window `MINT_LOCK_BLOCKS` (§3.6); their semantics are specified in §3. The planner stores *no* default parameters of its own: the entire configuration vector above is supplied by the pool creator at `newPool` (in the `PoolImmutables` argument, together with κ , the fees, and the protocol-fee settings) and captured as on-pool immutables — a deployed pool’s parameters do not change. The deployer enforces hard caps at construction time:

9.1 Mutable State on a Live Pool

The only mutable state on a deployed pool, post-`newPool`, is:

- **protocolFeeAddress** — recipient of accrued protocol fees, settable by the pool owner via `setProtocolFeeAddress`. Zero address is rejected when $\phi > 0$. The recipient is read at `collectProtocolFees` call time (see §6).

Table 2: Hard caps enforced at `PartyPlanner.newPool`

Parameter	Bound
Per-asset swap fee f_i	$< 10,000$ ppm ($< 1\%$)
Protocol fee share ϕ	$< 300,000$ ppm ($< 30\%$)
Asset count n	$2 \leq n \leq 383$
κ	> 0 (Q64.64)
Per-token base β_i	> 0
τ (MINT_DEVIATION_PPM)	$< 1,000,000$ ppm ($< 100\%$)
s (EMA_SHIFT_BLOCKS)	$1 \leq s < 64$
$\Gamma_{\max}$ (MAX_GAMMA_PER_WINDOW_PPM)	> 0
ℓ (MINT_LOCK_BLOCKS)	$\leq 50,400$ (one week of L1 blocks)

- Two-step ownership handoff via the `Ownable2Step` surface.
- The irreversible `kill()` flag.

9.2 Emergency Kill Switch

The pool owner may call `kill()`, after which the following state-mutating entry points revert: `swap`, `mint`, `swapMint`, `burnSwap`, and `initialMint`. The action is one-way; there is no `unkill`. Both `burn` (proportional LP redemption) and `collectProtocolFees` (sweep of already-accrued protocol fees to the configured `protocolFeeAddress`) remain operational on a killed pool by design: the proportional-redemption guarantee preserves LP exit under any kill scenario, and continued availability of `collectProtocolFees` ensures already-accrued protocol-fee balances are not stranded. `kill()` is intended for incident response if a critical vulnerability is discovered post-deployment; it is disjoint from the immutable parameter set.

9.3 What the Operator Cannot Do

The operator (planner owner and pool owner) explicitly *cannot*: mint LP out of band, burn user LP, change κ or any fee parameter on a live pool, pause user-side burns, blacklist or seize user balances, upgrade the pool implementation, renounce ownership, or move funds out of the pool directly. The full inventory of admin powers (CAN and CANNOT) is documented in `doc/security/admin-powers.md`.

10 Deployment Guidelines

10.1 Operator Workflow

The recommended end-to-end deployment workflow is:

1. **Token vetting.** Run `bin/validate-token <addr>` against every candidate token (see `doc/security/trusted-deployer-policy.md` for the validator policy). Resolve every `WARN` off-chain; `FAIL` blocks listing. Cross-check with `slither-check-erc <addr> ERC20-erc-conformance`. Record verifications in a listing record.

2. **Token-set composition.** Choose the basket of assets so the pool sits in a low-LVR regime (§10.2) — prefer correlated assets, group natural baskets, and avoid mixing high-volatility singletons with otherwise correlated sets.
3. **Parameter selection.** Choose κ (§10.3), per-asset fees, the σ_{swap} gate triple $(\tau, s, \Gamma_{\text{max}})$ (per the per-family table in `doc/rate-limited-mints.md`), and bases (§10.4).
4. **Initial liquidity.** Fund the deployer with `initialDeposits[i]` for each token; approve the planner.
5. **Deploy.** Call `PartyPlanner.newPool(...)`. The planner deploys the pool via `CREATE2`, pulls initial liquidity from the operator-specified `payer`, and invokes `initialMint` on the new pool, minting the operator-chosen initial LP supply $L^{(0)}$ (defaulting to 10^{18} if zero is passed) to `receiver`.
6. **Verify.** Confirm the pool address, `LMSR()` state, and `balances()` match expectations. Subscribe to the `PartyStarted` event for the indexer.

10.2 Token-Selection Criteria: Correlated-Asset LVR Reduction

The operator’s choice of which tokens to bundle into a single pool is a first-class design lever, separate from the per-asset fee and κ calibration. The load-bearing criterion is correlation structure across the basket, because LMSR’s multi-asset geometry exhibits a *cross-correlation discount* on the loss-versus-rebalancing (LVR) rate that is absent from separable bilateral CFMMs.

The discount. Olson & Claude Opus (2026, in-tree `doc/lvr-multiasset-lmsr.pdf`) derive the instantaneous LVR rate for the Hanson multi-asset LMSR in closed form. Writing P_i for external prices of N risky assets, $S = \sum_i P_i$, b for the LMSR liquidity parameter, and Σ for the return-covariance matrix, the rate decomposes as

$$\ell(P) = \underbrace{\frac{b}{2} \sum_i \sigma_i^2 P_i}_{N \text{ independent two-asset LMSR rates}} - \underbrace{\frac{b}{2(1+S)} P^\top \Sigma P}_{\text{cross-correlation discount}}.$$

The discount is non-negative for any positive-semidefinite Σ and grows with the variance of the value-weighted basket return: the more correlated the basket, the more cross-correlation cancels relative-mode arbitrage. For perfectly correlated equal-volatility assets the relative-mode contribution vanishes entirely and an $(N+1)$ -asset balanced pool collapses to the LVR of a single two-asset LMSR with matched b and matched risky notional — the multi-asset construction buys $N(N-1)/2$ pairwise quotes at the LVR cost of one. A pool of perfectly correlated assets with no numéraire has zero LVR.

Operator implication. Token selection should treat correlation as a primary criterion, not a secondary one. Concretely:

- **Prefer correlated baskets.** A liquid-staking-ETH bundle (stETH, rETH, ETH), a stablecoin bundle (USDC, USDT, DAI), a wrapped-BTC bundle (wBTC, cbBTC, tBTC), or any sector-correlated long-tail set will retain more value against the same arbitrage pressure than the same tokens spread across disjoint pairwise pools.

- **Avoid mixing.** A single highly-volatile uncorrelated asset inserted into an otherwise correlated basket re-introduces the relative-mode LVR contribution between that asset and every other slot; it is usually better to deploy the outlier in its own pool (or pair) than to dilute the correlation benefit of the surrounding basket.
- **Group by numéraire vs. risky.** A no-numéraire pool of correlated assets is the extreme of the discount; if a numéraire is included, the common-mode numéraire term dominates and the LVR floor is set by the basket-vs-numéraire variance.

The LVR paper’s closed-form decomposition gives the operator a quantitative tool to compare candidate baskets before deploying: compute $P^\top \Sigma P / (\sum_i \sigma_i^2 P_i)$ on a candidate composition, larger is better. The same calculation is the input to the κ and C calibration steps in §10.3–§10.5.

10.3 Choosing κ

κ sets the liquidity parameter $b = \kappa S(\mathbf{q})$ and so controls the trade-off between depth and drain-resistance. A higher κ deepens the pool — flatter price curve, tighter quotes, less slippage per trade — but lowers the external single-asset price move the pool absorbs before that slot drains; a lower κ steepens the curve so the pool re-prices through larger moves without draining. The maximum external single-asset price move before that slot drains is

$$R_{p,\max} = \exp\left(\frac{1}{\kappa(n-1)}\right). \quad (13)$$

For multi-asset pools the same $\kappa S(\mathbf{q})$ liquidity budget is shared across n slots, so single-slot divergence drains faster as n grows.

Calibration is simulation-driven, not table-driven. κ is not read off a generic by-asset-class table. It is tuned per pool with an off-chain analyzer (not part of the public repository) that simulates arbitrage actors against historical solver order flow (CoW) on the pool’s *fixed* token composition and fee schedule, sweeping κ for the value that maximizes LP yield without draining all but one slot. Because the fee schedule is an input to that simulation, **fees are chosen first** (§10.5) and κ is tuned against them. The two production pools land far apart: the volatile OG basket uses $\kappa = 0.2$ (a steep curve that survives large blue-chip and long-tail moves), while the correlated Peg Party stablecoin basket uses $\kappa = 5$ (a deep, tightly-quoting curve that drains a slot only on a genuine depeg — the desired “toxic flow stays out” behavior for assets assumed to hold their peg). See `analyzer/og-pool.md` and `analyzer/stable-pool.md`.

10.4 Bases β_i and Market-Calibrated Initialization

Each per-token base β_i converts raw token units to internal Q64.64 units: $q_i = \text{balance}_i / \beta_i$. The bases are not chosen freely — they are fixed by the `initialDeposits` array passed to `PartyPlanner.newPool`, in which β_i equals the per-token deposit amount that the deployer commits at `initialMint`. Because the kernel’s internal coordinate is $q_i = \text{balance}_i / \beta_i$, this means *every slot initializes at $q_i = 1.0$* regardless of the underlying token’s raw-unit magnitude or USD price.

Market-calibrated initial deposits. The deployment script that prepares the `initialDeposits` array is external to the pool repository and operates as follows:

1. Query realtime market prices for every candidate asset at deployment time from an off-chain reference (centralized exchange feed, oracle aggregator, etc.).
2. Compute a per-token initial deposit amount such that the *USD value* of each token’s initial deposit is equal across all slots in the new pool.
3. Pass that array to `newPool` as `initialDeposits`; the planner forwards it as the pool’s immutable β vector and pulls the matching amounts at `initialMint`.

The result is a pool whose initial internal state has $q_i = 1.0$ for every i , with the internal 1:1 marginal price between any two slots matching the slots’ realtime external market price ratio at deployment. The pool is launched in price-equilibrium with the external market.

Mis-calibration is an immediate-arbitrage failure mode. If the deployer’s initial-deposit ratios do *not* match the realtime external prices at deployment, the pool launches in disequilibrium: the internal 1:1 marginal price differs from the external market price, and arbitrageurs can extract the resulting spread immediately at the initial depositor’s expense. The pool itself has no internal way to detect this (it has no oracle); enforcement is entirely a property of the deployment script. The operator’s discipline is to run the calibration script as close to the deployment transaction as practical and to abort if intermediate price drift exceeds a small tolerance.

Why this works. Equal-USD-value deposits give every q_i identical magnitude in internal units, which:

- Maximizes precision headroom across natural drift.
- Aligns the spot price near 1:1 in internal units (true at the instant of launch, drifting thereafter with external prices).
- Sits the pool well inside the kernel’s `EXP_LIMIT` envelope.

Bases are immutable; revising them requires deploying a new pool.

10.5 Fee Calibration

Per-asset fees use a $C \cdot \sigma$ **method**: within a given pool, the fee rate for asset i is set proportional to a volatility estimate σ_i for that asset, scaled by a per-pool constant C . USDC anchors the schedule at 1.0 bps; every other listed asset’s fee is set so that f_i/σ_i matches USDC’s $f_{\text{USDC}}/\sigma_{\text{USDC}}$, then scaled by C . The scaling constant C is *not* protocol-wide — it is chosen per pool from competitive analysis of the venues that quote the same or comparable assets.

Bridge surcharge. Wrapped-bridged assets (e.g. `wSOL`) carry an additional heuristically-applied fee component on top of the $C\sigma$ baseline. The surcharge reflects the bridge-trust risk premium: an LP holding a wrapped asset bears the bridge’s solvency and operational risk in addition to the underlying asset’s market risk, and the fee schedule compensates them for that.

Step 1: calibrate C from per-pool competitive analysis. C is set as a first step, before any κ tuning, by reference to the fees charged by the largest-TVL competing venues on the pool’s asset composition. C is raised to the largest value that still “just undercuts” the prevailing market rate on the relevant pairs — high enough to extract meaningful fee revenue from organic flow, low enough that fee-sensitive takers prefer Liquidity Party to the competing venue. Once chosen, C is fixed for the rest of the calibration of *this pool*; a different pool composition may select a different C .

Reading the competing venue’s true market rate. Several mainstream AMMs (Uniswap v3 is the canonical case) expose only a coarse set of fee tiers — typically $\{1, 5, 30, 100\}$ bps — so the tier a competing pool sits on is not always a clean signal of the prevailing market rate. The operator’s job is to recover the *economic* market fee from the deployed tier by judging the tier’s TVL share against the competing tiers on the same pair, and by backing out any structural cost that the tier price covers but that does not apply to a marginal Liquidity Party depositor. The canonical example is WBTC/USDT, which clears at the 30 bps tier on Uniswap v3 despite the underlying asset volatility implying a much smaller economically-fair rate; the operator interprets the residual as the LP’s structural cost of the WBTC redemption window (roughly ~ 20 bps of one-way redemption friction that WBTC LPs must price in to avoid losing on forced-redemption events). That structural cost is absent for a marginal Liquidity Party LP, so the operator subtracts it discretionarily to arrive at the volatility-matched market rate used in the $C \cdot \sigma$ fit for the new pool. Similar discretionary adjustments apply to any competing venue whose deployed fee covers redemption, custody, or other structural LP costs that do not transfer to a Liquidity Party deposit.

Step 2: optimize κ at fixed C . With C fixed, κ is tuned against a historical simulation over at least one year of price and flow data. The simulation operates two flow streams against a candidate pool:

1. A *Binance-connected arbitrageur* able to take infinite liquidity at the Binance inside quote plus 2 bps. This is an upper bound on real arbitrage activity — the inside is thin in practice, so a real arb cannot take infinite size at the inside plus 2 bps. The simulation therefore over-estimates arbitrage pressure on the pool, biasing κ toward conservative (LP-favoring) values.
2. *Organic flow* derived from on-chain trade data, specifically CoW Swap auction participant flow over the same historical window. This is real, fee-sensitive user demand.

The objective is the LP’s terminal net return: fees collected minus LVR-style adverse-selection cost minus realized IL on slot drift, measured at the simulated horizon. κ is chosen to maximize that net at the fixed C .

The procedure — C from per-pool competitive analysis, then κ from LP-net-return optimization at the chosen C — decouples the two parameters: C reflects the pool’s competitive position and κ reflects historical LP economics under that competitive position. Both are re-run for each new pool whose asset composition or competitive landscape materially differs from a previously calibrated pool; reuse is allowed only when the new pool genuinely sits in the same competitive envelope.

10.6 Native-Asset Wrapping

If the pool will accept native ETH (or chain-native equivalent), one of the pool’s tokens must be the canonical wrapper (WETH9 on mainnet, the equivalent on each L2). The wrapper address is

passed via the planner constructor and is the only address authorized to push raw ETH into the pool through `receive()`.

11 Security Review Process

11.1 Layered Defense

The protocol applies a defense-in-depth posture (`doc/security/threat-model.md §1`):

- **Permissioned deploy** as the primary defense against malicious tokens. `PartyPlanner.newPool` is `onlyOwner`, indefinitely. The operator is the trust anchor for token vetting.
- **Runtime guards.** `nonReentrant` on every state-mutating entry point, `killable` on every entry point except `burn` and `collectProtocolFees`, payer-gates on the `APPROVAL` and `PREFUNDING` funding paths, deploy-time delta-equality on the initial-deposit transfer, EIP-712 witness binding on every `Permit2` path.
- **Off-chain validation.** `bin/validate-token` + `slither-check-erc` are mandatory pre-listing. The validator emits `PASS/WARN/FAIL`; `WARN` requires off-chain verification and a recorded listing-record entry, `FAIL` blocks the listing entirely.
- **Monitoring + kill switch.** Off-chain drift monitoring with documented yellow/red thresholds; `kill()` as the last-resort lever.

There is no on-chain price oracle dependency; pricing is intrinsic to the LMSR kernel.

11.2 Internal Review Artifacts

The repository carries the following internal review artifacts that gate launch (see `doc/security/`):

- `threat-model.md` — threat model, actors, fund-bearing assets, attack vectors, invariants. v1 pre-deploy.
- `checklist.md` — the security review checklist (14 sections, 84 rows). **Every row is closed.** Engineering and code-correctness rows are recorded as `MITIGATED` (closed in source) or `DOCUMENTED` (closed by docs); the external audit is complete (§11.4), leaving one operational row — the migration to a multisig admin key — recorded as `DEFERRED WITH TRIGGER` at a named TVL threshold, with compensating mitigations specified in `threat-model.md §11`. No row is open or unresolved.
- `asset-authority-matrix.md` — (actor × asset × function) → gate @ file:line, the per-cell authorization matrix.
- `admin-powers.md` — CAN/CANNOT inventory of every `onlyOwner` surface, with trapped-funds analysis.
- `trusted-deployer-policy.md` — operator obligations, the token-class denylist (fee-on-transfer, rebasing, ERC-777-style hook tokens, multi-address proxy, mutable governance), the listing checklist.
- `economics-audit.md` — the economics-side review of the fee structure, single-asset round trips, and LP-favor invariants.

- `storage-layout-audit.md` — audit of the C3-linearized storage layout used under `DELEGATECALL`.
- `unchecked-blocks-audit.md` — justification for every `unchecked` block in `src/`.
- `differential-review.md` — differential review of the block-anchored *b* kernel and the wedge defense.
- `tooling-runbook.md` — Slither, Mythril, and `forge-test` invocation reference.
- `user-allowance-guidance.md` — guidance for integrators on the Concierge approval surface and Permit2 usage.

11.3 Tooling

- **Foundry / `forge-test`.** The in-tree test suite carries the cost-preservation, kernel-no-overshoot, gate-tripping, rate-limit-boundary, min-clamped-burn, JIT-closure, wedge-attack, and (actor $\times$ asset) gating tests that close the checklist. Coverage is collected with `forge coverage -ir-minimum -no-match-contract "ContractSize|GasTest"`.
- **Slither.** Static analysis with the project's `slither.config.json`. False-positive suppressions are committed alongside the code they suppress and are reviewed in the checklist.
- **Token validator.** `bin/validate-token` probes a candidate token across the property axes that the protocol cannot defend against at runtime (fee-on-transfer, rebasing, hook callbacks, multi-address proxy, mutable governance). See `doc/security/trusted-deployer-policy.md`.

11.4 External Audit

The codebase was audited in its entirety over 15 full working days by Jay Joshi. All findings were mitigated.

11.5 Bug Bounty

Liquidity Party is launching as a pre-revenue protocol and is not in a position to commit to a formal bug bounty program prior to launch. Per `SECURITY.md`, severity-graded discretionary thanks are extended for responsible reports during the interim period. The intent is to launch a formal bounty (Immunefi Boost or equivalent) once launch metrics justify the spend. This section will be updated once a bounty program has been established.

11.6 Vulnerability Disclosure

Security reports are routed privately by email to `tim@dexorder.com` or via direct message to `@LiquidityParty` on X. Public GitHub issues for security vulnerabilities are explicitly discouraged. The disclosure SLA is acknowledgement within 48 hours and a coordinated disclosure timeline scaled to severity. The reporter is credited at their option once the issue is mitigated. The full policy lives in `SECURITY.md`.

11.7 Out of Scope

Per `SECURITY.md`, the following are out of scope:

- Test fixtures, mocks, and helpers under `test/`.
- Issues that require a compromised admin key as a precondition. Admin keys can `kill()` a pool and redirect protocol fees, but cannot reach LP reserves; this is documented as accepted risk in `doc/security/threat-model.md §11.N7` and `doc/security/admin-powers.md`.

12 Risk Management

12.1 Token-Slot Drainage Is Expected LP Behavior

Token slots in a Liquidity Party pool will drain. This is normal, expected, and intentional. (We use “token slot” for a pool’s holding of one of its constituent tokens — the position the pool maintains in, for example, the WBTC token within a WBTC/USDC/ETH pool.) The protocol’s deployment strategy is to calibrate (C, κ) for maximum LP net return after impermanent loss (§10.5). Optimizing for net LP return implies that token slots whose underlying asset experiences a large persistent price move *will* reach the drained state $q_k \approx 0$ before the pool internalizes the full move. This is the same trade-off every concentrated-liquidity AMM makes.

Analogy: Uniswap v3 range positions. A Uniswap v3 LP who posts liquidity over a finite price range collects fees only while the spot price is inside the range. If the price moves through the upper end of the range, the LP ends up holding 100% of the “cheaper” asset — their position has converted entirely to the appreciated asset’s pair-leg, exactly as if they had executed a market sell at every intermediate price along the range. Range LPs accept this as the fundamental staking trade-off of v3: deeper liquidity inside the range in exchange for full conversion at the boundary. Drainage in Liquidity Party is the multi-asset analogue. A token slot is implicitly “range-staked” against the rest of the basket: while the relative price stays within the pool’s depth envelope, the slot earns fees on the LMSR quote; if the external market moves the slot’s price far enough that the pool can fund cheaper-than-external swaps until the slot empties, arbitrageurs will drain it. The maximum single-token-slot external move the pool can absorb before drain is

$$R_{p,\max} = \exp\left(\frac{1}{\kappa(n-1)}\right) \quad (14)$$

(§10.3); past that, the token slot empties and the basket settles in the LP’s hands without that asset.

What the LP retains after a drain. A drained token slot does not impair the LP’s redemption rights. Proportional **burn** continues to work and returns the LP their share of every token slot that still has reserves. Single-asset **burnSwap** into the drained token returns nothing for that token but treats the drained slot as zero-contribution rather than reverting; **burnSwap** into a non-drained token still works. The LP’s economic position after a token-slot drain is equivalent to holding the surviving basket — exactly the position a v3 LP holds after a one-sided range exit.

Industry framing. Drainage in this sense is the industry-standard cost of providing concentrated liquidity. Liquidity Party does not promise to retain every token under every market scenario; it promises (a) pro-rata redemption of whatever the pool currently holds, (b) LP-favoring rounding and approximation at every step, and (c) a fee schedule calibrated against historical data with the goal of anticipated LP net return inclusive of expected drainage events. The historical calibration is a best-effort estimate; past flow and price dynamics are not a guarantee of future LP returns, and the realized net is subject to the market conditions that unfold after deployment. LPs participating in pools that hold uncorrelated or highly volatile assets should expect drain events on individual token slots and should size their participation accordingly.

12.2 LP Loss Bound

Under constant b , classical LMSR admits a worst-case loss bound of $b \ln n$ in the payoff numéraire. With $b(\mathbf{q}) = \kappa S(\mathbf{q})$, the per-state bound scales with the current size metric: instantaneous worst-case loss $b(\mathbf{q}) \ln n = \kappa S(\mathbf{q}) \ln n$, or $\kappa \ln n$ per unit of $S(\mathbf{q})$. Because b varies with state, global worst-case loss along an arbitrary path depends on how $S(\mathbf{q})$ evolves; the proportionality clarifies how risk scales with deployed liquidity.

12.3 Liquidity-Manipulation Resistance

Every operation that changes $S(\mathbf{q})$ is priced through the same LMSR potential that prices ordinary swaps:

- Proportional mint and burn are exact scalings: $\mathbf{q} \mapsto (1 + \alpha)\mathbf{q}$ implies $b \mapsto (1 + \alpha)b$ and every pairwise marginal ratio is invariant. A proportional mint followed by its exact inverse returns the pool to the same state with no net transfer.
- Single-asset mint and burn (`swapMint`, `burnSwap`) decompose into a proportional rescaling composed with a bundle of LMSR swaps priced by the same convex potential. The cost of changing $S(\mathbf{q})$ is the sum of kernel-priced components; there is no off-market “free” lever to change b .

Because $C(\mathbf{q})$ is convex, the net kernel cost of any closed cycle of allowed operations sums to zero in the fee-free idealization; any practical implementation cost (fees, LP-favor rounding) makes the cycle strictly loss-making for the attacker. The block-anchored b (§2.4) ensures that every leg of every intra-block sequence of `swap`, `swapMint`, and `burnSwap` prices against the same convex potential; combined with the wedge defense (§3) on the mint side, adversarial closed cycles run on already-imbalanced pools cannot extract a single wei (§2.6, “Empirical zero-extraction”).

12.4 Numerical Envelope

The kernel’s cost-preservation residual sits below 10^{-9} relative across the production-relevant range of pool imbalances ($R_q \leq 10^7$) and degrades gracefully past that. The ABDK exp/ln round-trip noise floor is $\sim 2 \times 10^{-16}$ relative; production tolerances are bound at 10^{-9} , leaving seven orders of magnitude of headroom (see `security/drainage-guidelines.md` §3, §4).

12.5 Operational Mitigations

- User `minAmountOut` / `maxAmountIn` bounds on every state-mutating entry point.

- Capacity caps on outputs ($y \leq q_j$), enforced inside the kernel.
- Minimum-balance constraints on `initialMint` (every initial deposit must be strictly positive; $\beta_i > 0$ enforced at deploy time).
- Off-chain drift monitoring with yellow/red thresholds on the token-slot drain ratio $\min q_i / (S(\mathbf{q})/n)$ and the q-ratio $\max q / \min q$.
- `kill()` as the last-resort lever, preserving proportional burn for LP exit.

12.6 Out-of-Scope Token Classes

The protocol does not support, and the trusted-deployer policy **MUST NOT** list:

- Fee-on-transfer tokens.
- Rebasing tokens.
- ERC-777-style hook tokens, ERC-677 `transferAndCall` tokens, any token that callbacks during `transfer` or `transferFrom`.
- Multi-address proxy tokens.
- Tokens whose owner can later add fees, pause transfers, or blacklist addresses (mutable post-list governance).

A token that the validator clears today but later mutates into one of the above classes is a class-2 hazard; the response is `kill()`.

13 Deployment Surface

Initial deployment. The initial production deployment targets Ethereum mainnet. The first planned expansion, if launch metrics support it, is Base. The protocol is EVM-only — non-EVM chains are out of scope for the foreseeable future. Per-chain contract addresses (`PartyPlanner`, `PartyInfo`, `PartyConcierge`) are published in `deployment/liqp-deployments.json` in the repository. That file is the canonical record of deployed contract information and is updated at deploy time.

Operator key custody. The admin (planner owner / pool owner) key is held on a hardware wallet. No further detail of the operator’s internal key-management procedure is disclosed; the LP’s security guarantee does not depend on it (§5.4). The protocol is designed to be *resilient against an admin-key leak*: a leaked admin key cannot reach LP reserves under any operation in the contract surface. The worst case under a leaked key is denial-of-service (`kill()` called by the attacker) plus redirection of not-yet-collected protocol fees; LP `burn` remains operational and LPs retain their pro-rata share of reserves. The core team retains independent ability to invoke `kill()` on the planner-owned pools as a final defensive measure if the admin key is compromised.

Indexer. Liquidity Party does not host a public subgraph at this time. Token and pool metadata is published as a static `deployment/metadata.json` artifact, which is the canonical source for that metadata; `deployment/liqp-deployments.json` holds only the deployed contract information.

Integrators that need event-level data should consume the canonical event set directly from the chain: `PartyStarted`, `Mint`, `Burn`, `Swap`, `SwapMint`, `BurnSwap`, `ProtocolFeesCollected`, `Killed`.

14 Disclaimers

Issuer. Liquidity Party is operated by Dexorder Trading Services, Ltd., a company incorporated in the British Virgin Islands under registration number 2160219.

Access restrictions. The Liquidity Party front-end blocks access from addresses appearing on the U.S. Office of Foreign Assets Control (OFAC) Specially Designated Nationals (SDN) and consolidated sanctions lists. The on-chain smart contracts are permissionless to interact with from any address; the access restriction is enforced at the front-end and any sanctioned address that interacts directly with the contracts does so outside the front-end’s terms of service.

No investment advice. This whitepaper is for informational purposes only. It does not constitute investment advice, a solicitation of any security, or a representation as to the suitability of the protocol for any user, jurisdiction, or purpose. Past performance of the simulation environment described in §10.5 is not indicative of future LP returns.

No representation as to tax treatment. The tax treatment of LP participation, swap activity, and protocol-fee distribution is jurisdiction-specific and depends on facts and circumstances outside the protocol’s control. Users are responsible for their own tax compliance.

Forward-looking statements. Statements in this whitepaper regarding future deployments (Base expansion, formal bug bounty, metadata index), future audit report publication, and future protocol behavior are forward-looking. They reflect current intent and are not commitments.

Risk acknowledgement. Liquidity provision in Liquidity Party pools is subject to impermanent loss, token-slot drainage loss (§12.1), smart-contract risk, oracle-independent depegging of constituent assets, bridge risk on wrapped-bridged assets, and the residual risks documented in §12. LPs should understand these risks before depositing.

References

- Hanson, R. (2002). *Logarithmic Market Scoring Rules for Modular Combinatorial Information Aggregation*. mason.gmu.edu/~rhanson/mktscore.pdf
- Othman, A., Pennock, D., Reeves, D., Sandholm, T. (2013). *A Practical Liquidity-Sensitive Automated Market Maker*. [cs.cmu.edu/~sandholm/liquidity-sensitive market maker.EC10.pdf](https://cs.cmu.edu/~sandholm/liquidity-sensitive%20market%20maker.EC10.pdf)
- Olson, T., Claude Opus 4.7 (2026). *Loss-Versus-Rebalancing Reduced in Cross-Correlated Multi-Asset LMSR Pools*. doc/lvr-multiasset-lmsr.pdf (this repository).